Web Programming

Rodrigo Martins Pagliares

Last update: 11/01/2025

Web Programming

Rodrigo Martins Pagliares

JavaScript
Dynamic client-side scripting

JavaScript first steps

developer.mozilla.org

Get real-time assistance with your coding queries. Try AI Help now!

mdn web docs _ References Guides Plus Blog Play NEW AI Help BETA Theme Log in Get MDN Plus

Guides English (US)

Filter

HTML — Structuring the web

- Introduction to HTML
- Multimedia and embedding
- HTML tables

CSS — Styling the web

- CSS first steps
- CSS building blocks
- Styling text
- CSS layout

JavaScript — Dynamic client-side scripting

- JavaScript first steps
- JavaScript building blocks
- Introducing JavaScript objects
- Asynchronous JavaScript
- Client-side web APIs

Web forms — Working with user data

Learn web development

Welcome to the MDN learning area. This set of articles aims to guide complete beginners to web development with all that they need to start coding websites.

The aim of this area of MDN is not to take you from "beginner" to "expert" but to take you from "beginner" to "comfortable." From there, you should be able to start making your way, learning from the rest of MDN, and other intermediate to advanced resources that assume a lot of previous knowledge.

If you are a complete beginner, web development can be challenging — we will hold your hand and provide enough detail for you to feel comfortable and learn the topics properly. You should feel at home whether you are a student learning web development (on your own or as part of a class), a teacher looking for class materials, a hobbyist, or someone who just wants to understand more about how web technologies work.

Looking to become a front-end web developer?

We have put together a course that includes a
towards your goal.

Get started

In this article

- Where to start
- Topics covered
- Getting our code examples
- Contact us
- See also



How to uplevel your hybrid cloud connection on Equinix Metal using Equinix Fabric

Mozilla ads

https://tinyurl.com/mryx9yzw

What went wrong? Troubleshooting JavaScript

Lecture: What went wrong? Troubleshooting JavaScript
Module: JavaScript first steps

Filter

JavaScript — Dynamic client-side scripting

- JavaScript first steps
  - JavaScript First Steps
  - What is JavaScript?
  - A first splash into JavaScript
  - What went wrong? Troubleshooting JavaScript
  - Storing the information you need — Variables
  - Basic math in JavaScript — numbers and operators
  - Handling text — strings in JavaScript
  - Useful string methods
  - Arrays
  - Silly story generator
- JavaScript building blocks
- Introducing JavaScript objects
- Asynchronous JavaScript
- Client-side web APIs

Web forms — Working with user

What went wrong? Troubleshooting JavaScript

Previous Overview: First steps Next

When you built up the "Guess the number" game in the previous article, you may have found that it didn't work. Never fear — this article aims to save you from tearing your hair out over such problems by providing you with some tips on how to find and fix errors in JavaScript programs.

Prerequisites:	Basic computer literacy, a basic understanding of HTML and CSS, an understanding of what JavaScript is.
Objective:	To gain the ability and confidence to start fixing problems in your own code.

Types of error

Generally speaking, when you do something wrong in code, there are two main types of error that you'll come across:

- **Syntax errors**: These are spelling errors in
  at all, or stop working part way through — y
  messages too. These are usually okay to fix
  know what the error messages mean!
- **Logic errors**: These are errors where the syntax is actually correct but the code is not what you

In this article

- Types of error
- An erroneous example
- Fixing syntax errors
- A logic error
- Other common errors
- Summary
- See also

GitLab

Simplify software development with the One DevOps Platform. Start your free 30 day trial

https://tinyurl.com/mr27hw8c

Objective of the lecture

Objective: To gain the **ability** and confidence to **start fixing problems** in your own **code**.

Introduction

Introduction

- This **lecture** aims to **provide** you with some **tips** on **how** to **find** and **fix errors** in **JavaScript programs**.

Types of errors

Types of errors

- **There** are **two** main **types** of **errors**.
  1. **Syntax** errors.
  2. **Logic** errors.

Syntax errors

Syntax errors

- **Spelling errors** in your **code** that actually **cause** the **program not to run** at all, or **stop working** part way through.

- You will usually be **provided** with **some error messages**.

Logic errors

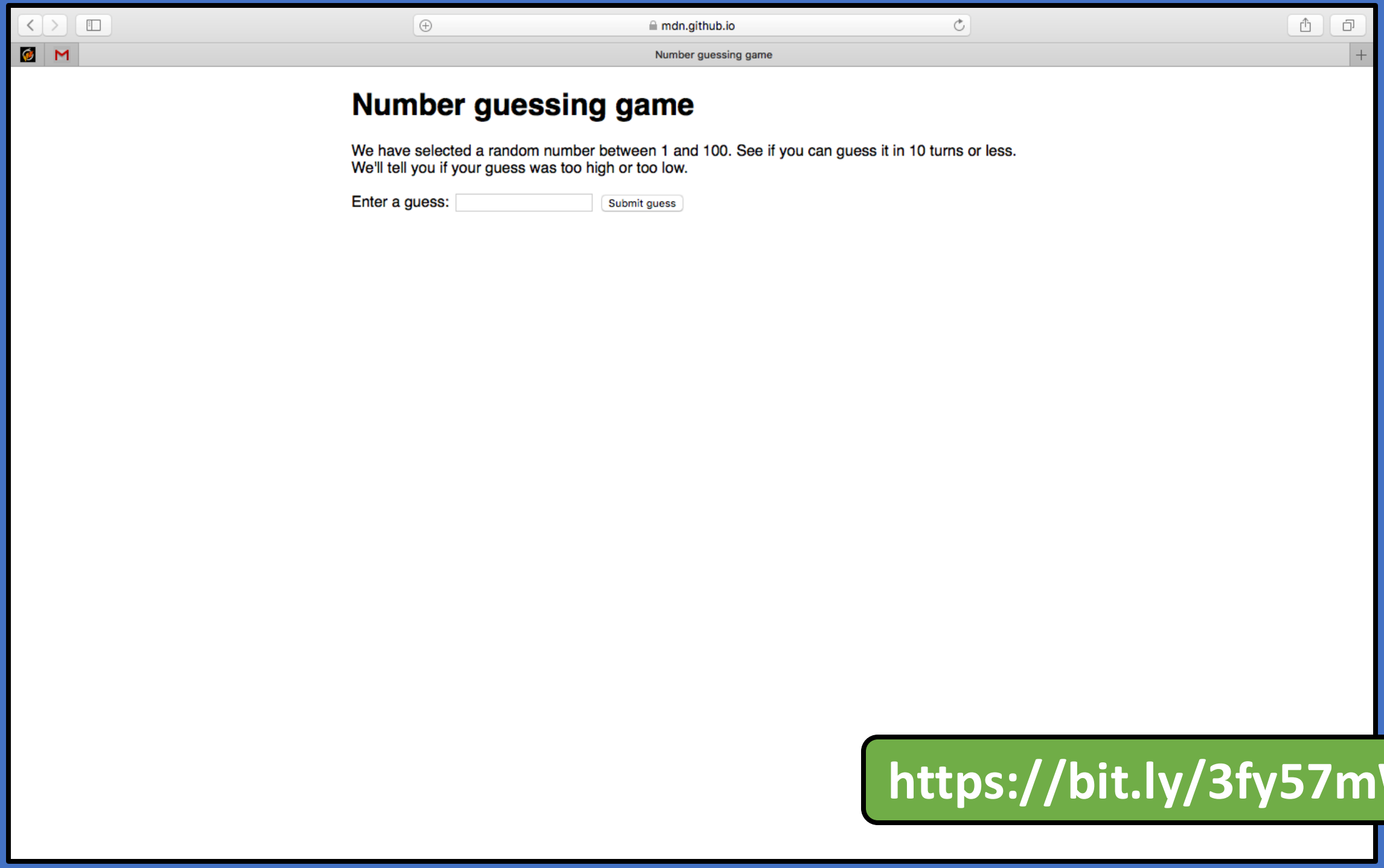
Logic errors

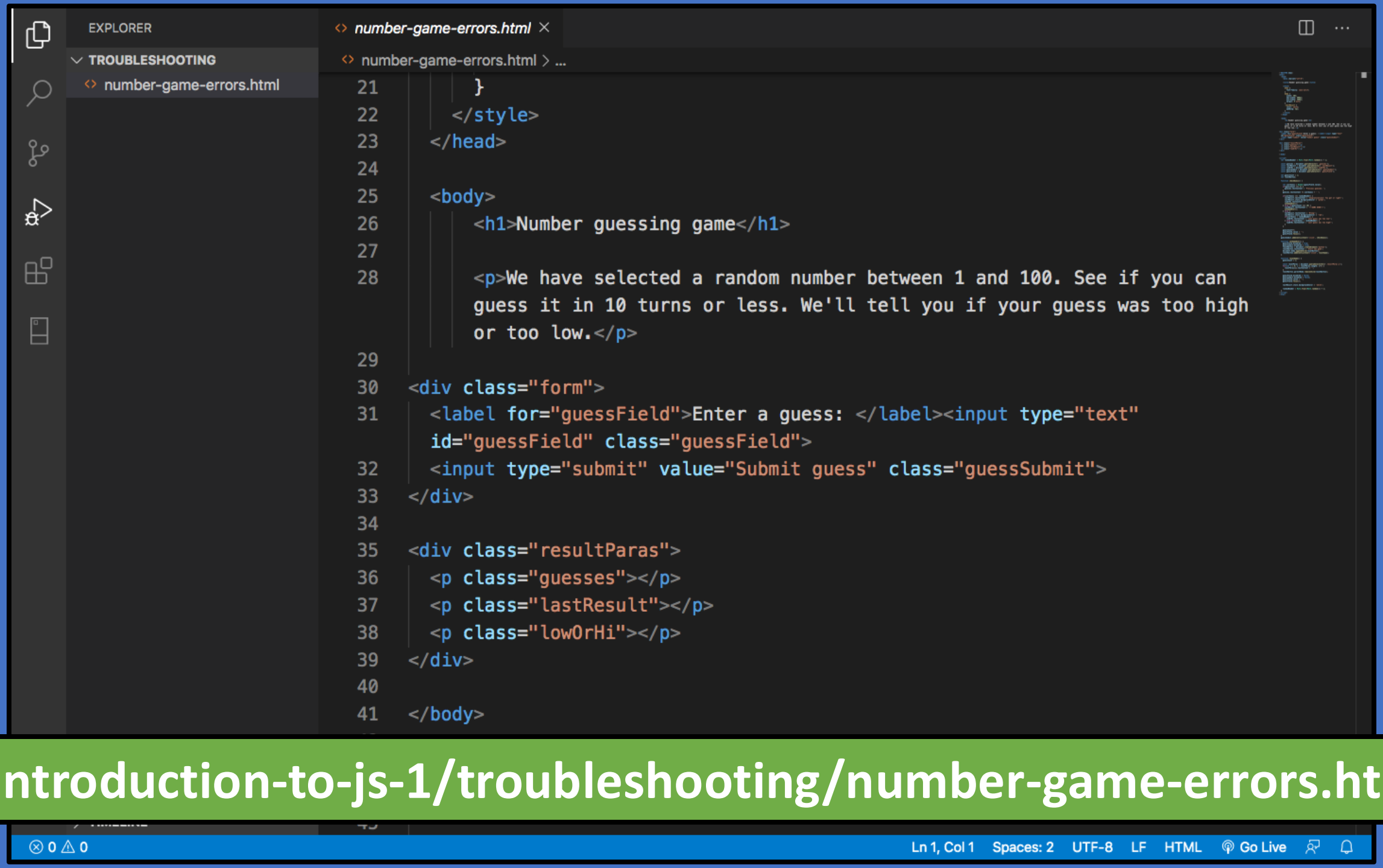
- The **syntax** is actually **correct but** the **code** is **not** what you **intended** it to **be**.
  - ✓Meaning that **program** runs **successfully** but gives **incorrect results**.
- Often **harder** to **fix** than **syntax errors**, as **there** usually **isn't** an **error message** to **direct you** to the **source** of the **error**.

An erroneous example

An erroneous example

- Let's **return** to our **number guessing game**.
  - ✓**Except** this **time** we'll be exploring a **version** that **has some deliberate errors introduced**.

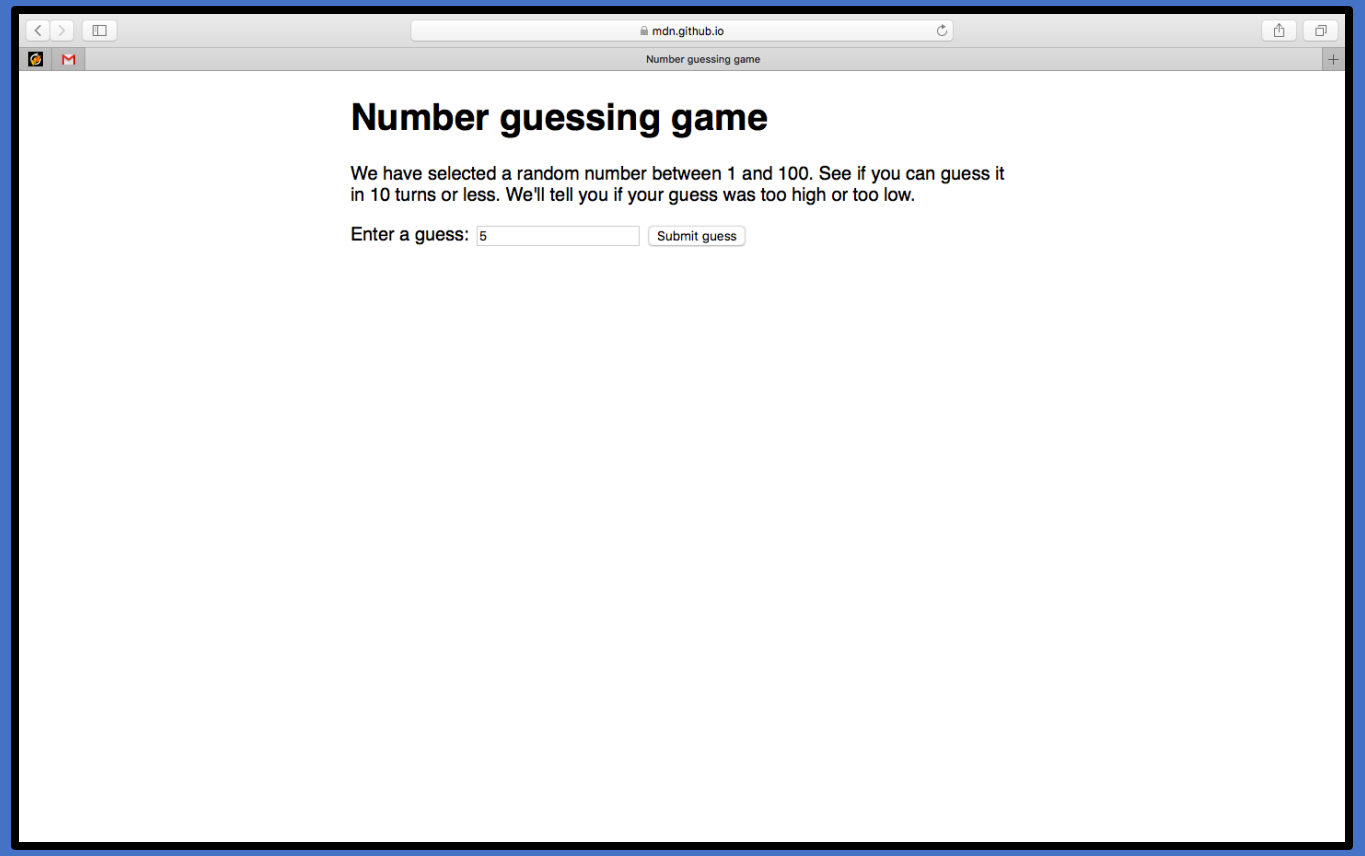

mdn.github.io
Number guessing game
Number guessing game
We have selected a random number between 1 and 100. See if you can guess it in 10 turns or less.
We'll tell you if your guess was too high or too low.
Enter a guess:
Submit guess
https://bit.ly/3fy57mW



introduction-to-js-1/troubleshooting/number-game-errors.html

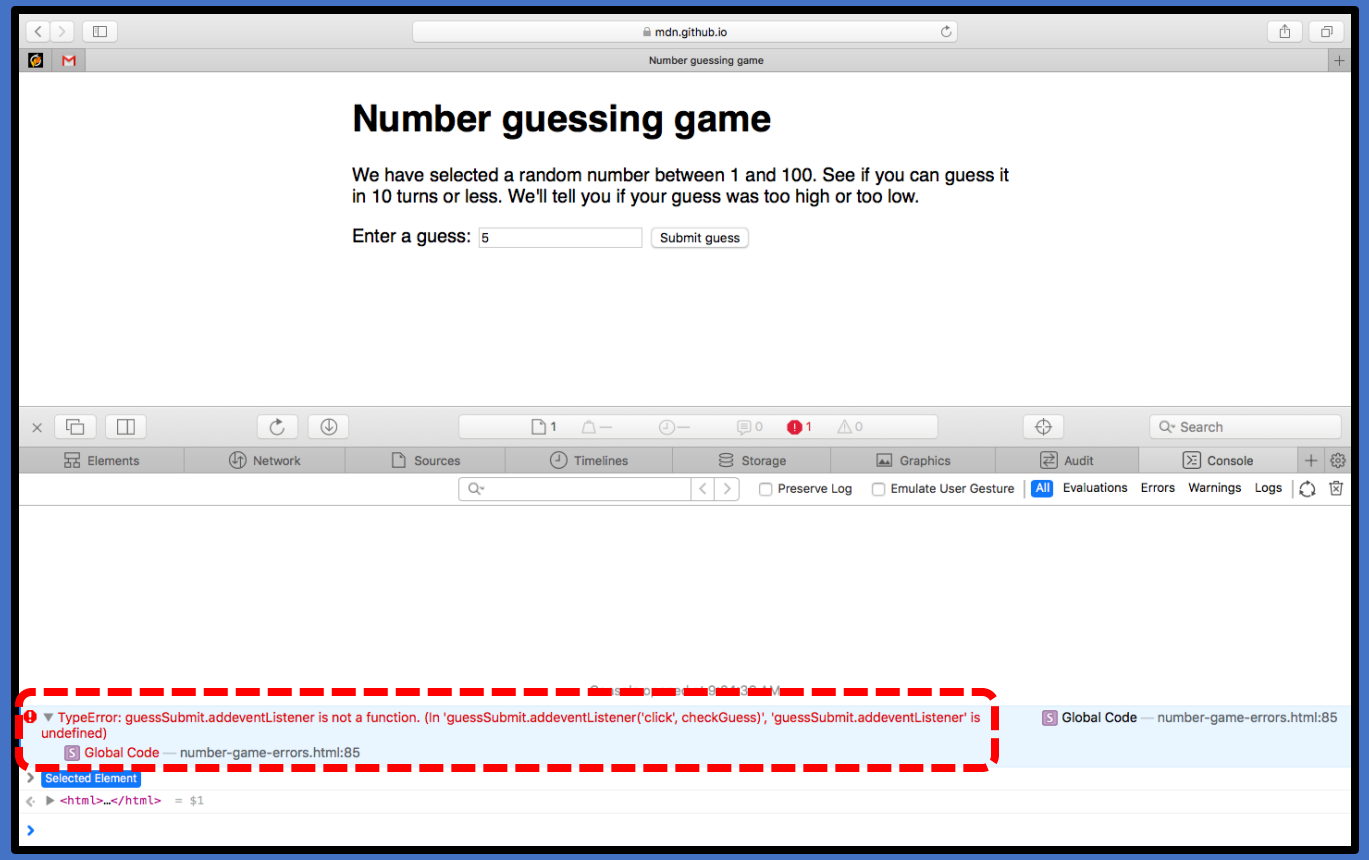
An erroneous example

- **Try playing** the **game**.
  - ✓You'll **notice** that when you **press** the "**Submit guess**" **button**, it **doesn't work**!
- **Open** the **developer console** to **see** if it **reports** any **syntax errors**, then **try to fix them**.

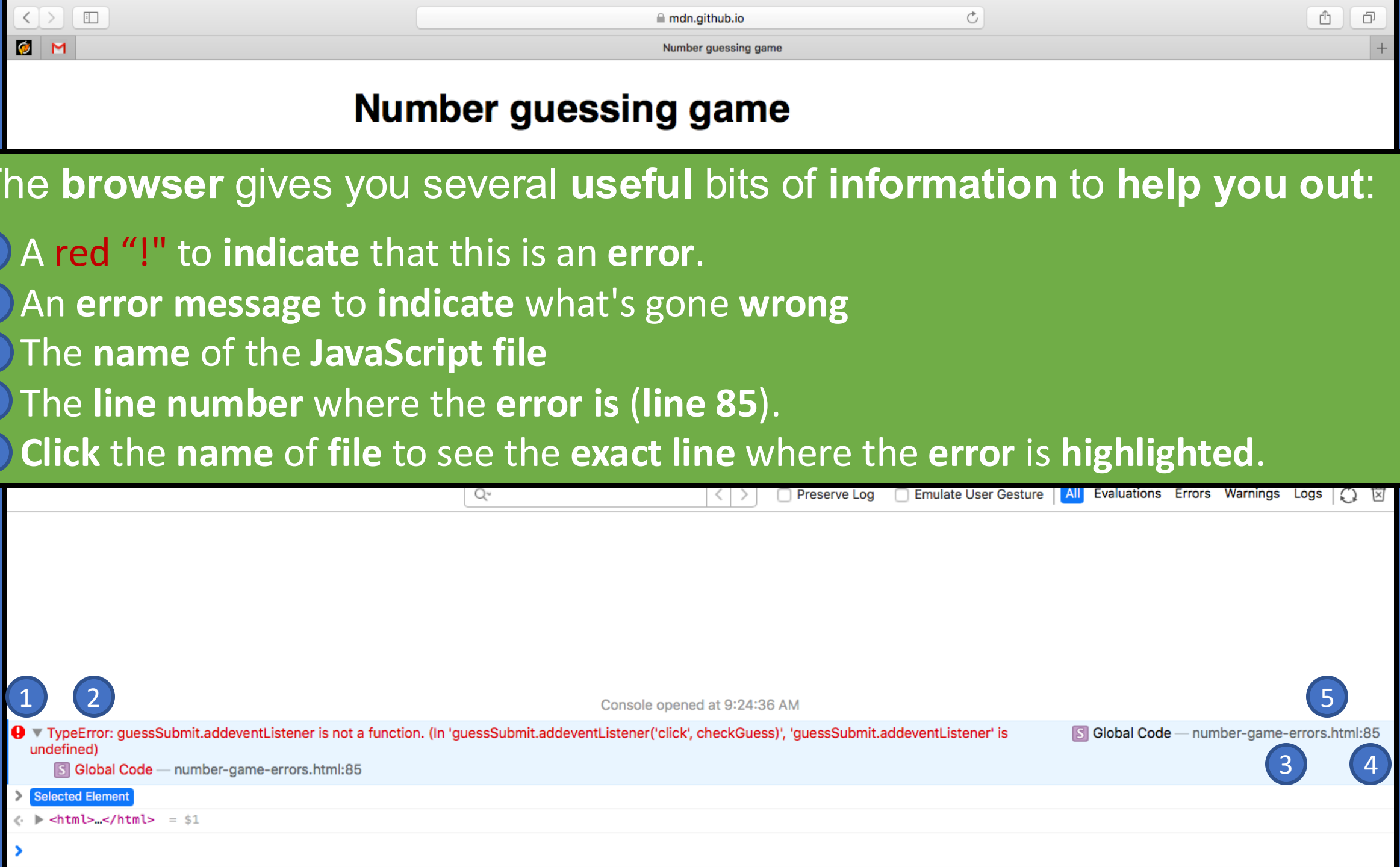


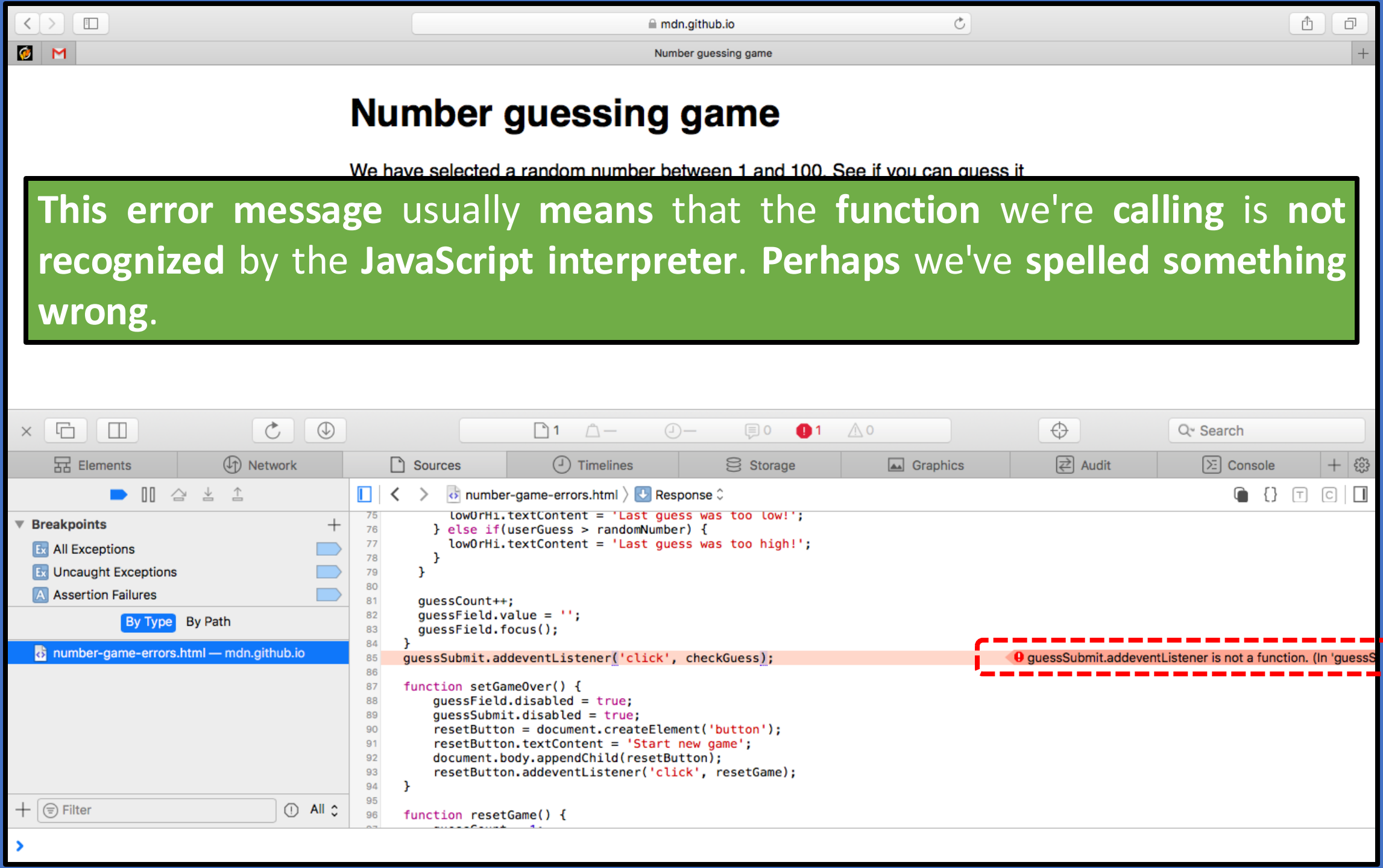
Fixing syntax errors

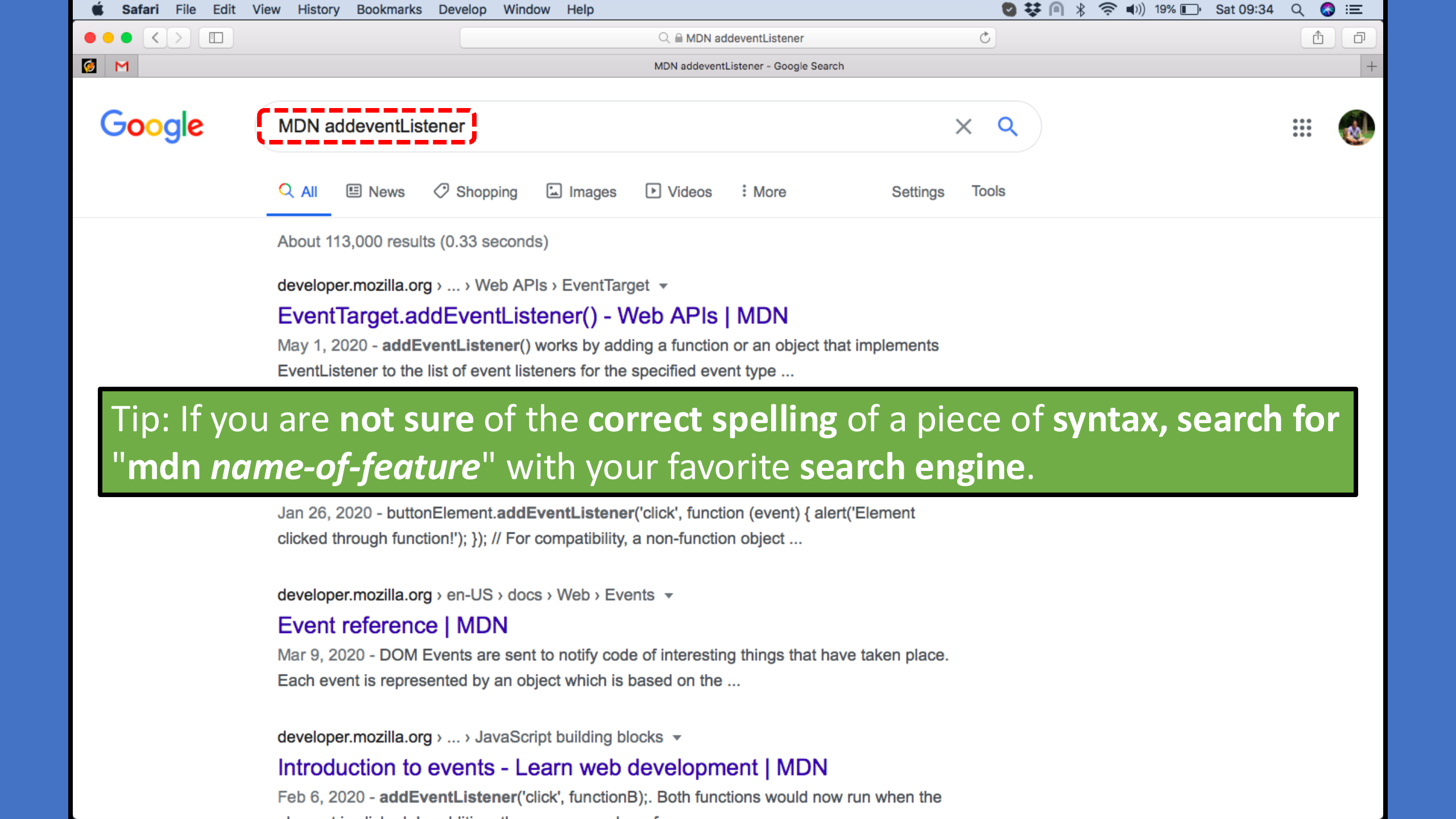
- The **developer tools JavaScript console** gives you **error messages whenever** a **syntax error** exists inside the **JavaScript** being fed into the **browser's JavaScript engine**.



Fixing syntax errors
(Round 1/3)


mdn.github.io
Number guessing game
Number guessing game
The browser gives you several useful bits of information to help you out:
1 A red "!" to indicate that this is an error.
2 An error message to indicate what's gone wrong
3 The name of the JavaScript file
4 The line number where the error is (line 85).
5 Click the name of file to see the exact line where the error is highlighted.
Preserve Log
Emulate User Gesture
All
Evaluations
Errors
Warnings
Logs
1
2
Console opened at 9:24:36 AM
5
TypeError: guessSubmit.addeventListener is not a function. (In 'guessSubmit.addeventListener('click', checkGuess)', 'guessSubmit.addeventListener' is undefined)
Global Code — number-game-errors.html:85
Global Code — number-game-errors.html:85
3
4
Selected Element
<html>…</html> = $1


mdn.github.io
Number guessing game
Number guessing game
We have selected a random number between 1 and 100. See if you can guess it
This error message usually means that the function we're calling is not recognized by the JavaScript interpreter. Perhaps we've spelled something wrong.
Search
Elements
Network
Sources
Timelines
Storage
Graphics
Audit
Console
Breakpoints
All Exceptions
Uncaught Exceptions
Assertion Failures
By Type
By Path
number-game-errors.html — mdn.github.io
number-game-errors.html
Response
lowOrHi.textContent = 'Last guess was too low!';
} else if(userGuess > randomNumber) {
lowOrHi.textContent = 'Last guess was too high!';
}
}
guessCount++;
guessField.value = '';
guessField.focus();
}
guessSubmit.addeventListener('click', checkGuess);
guessSubmit.addeventListener is not a function. (In 'guessS
function setGameOver() {
guessField.disabled = true;
guessSubmit.disabled = true;
resetButton = document.createElement('button');
resetButton.textContent = 'Start new game';
document.body.appendChild(resetButton);
resetButton.addeventListener('click', resetGame);
}
function resetGame() {
Filter
All


Safari File Edit View History Bookmarks Develop Window Help
19% Sat 09:34
MDN addeventListener
MDN addeventListener - Google Search
Google
MDN addeventListener
All News Shopping Images Videos More Settings Tools
About 113,000 results (0.33 seconds)
developer.mozilla.org › ... › Web APIs › EventTarget
EventTarget.addEventListener() - Web APIs | MDN
May 1, 2020 - addEventListener() works by adding a function or an object that implements EventListener to the list of event listeners for the specified event type ...
Tip: If you are not sure of the correct spelling of a piece of syntax, search for "mdn name-of-feature" with your favorite search engine.
Jan 26, 2020 - buttonElement.addEventListener('click', function (event) { alert('Element clicked through function!'); }); // For compatibility, a non-function object ...
developer.mozilla.org › en-US › docs › Web › Events
Event reference | MDN
Mar 9, 2020 - DOM Events are sent to notify code of interesting things that have taken place. Each event is represented by an object which is based on the ...
developer.mozilla.org › ... › JavaScript building blocks
Introduction to events - Learn web development | MDN
Feb 6, 2020 - addEventListener('click', functionB);. Both functions would now run when the

EventTarget.addEventListener()

On this Page

- Syntax
- Usage notes
- Examples
- Other notes
- Specifications
- Browser compatibility
- See also

The `EventTarget` method **`addEventListener()`** sets up a function that will be called whenever the specified event is delivered to the target. Common targets are `Element`, `Document`, and `Window`, but the target may be any object that supports events (such as `XMLHttpRequest`).

`addEventListener()` works by adding a function or an object that implements `EventListener` to the list of event listeners for the specified event type on the `EventTarget` on which it's called.

Related Topics

EventTarget

▾ Constructor

EventTarget()

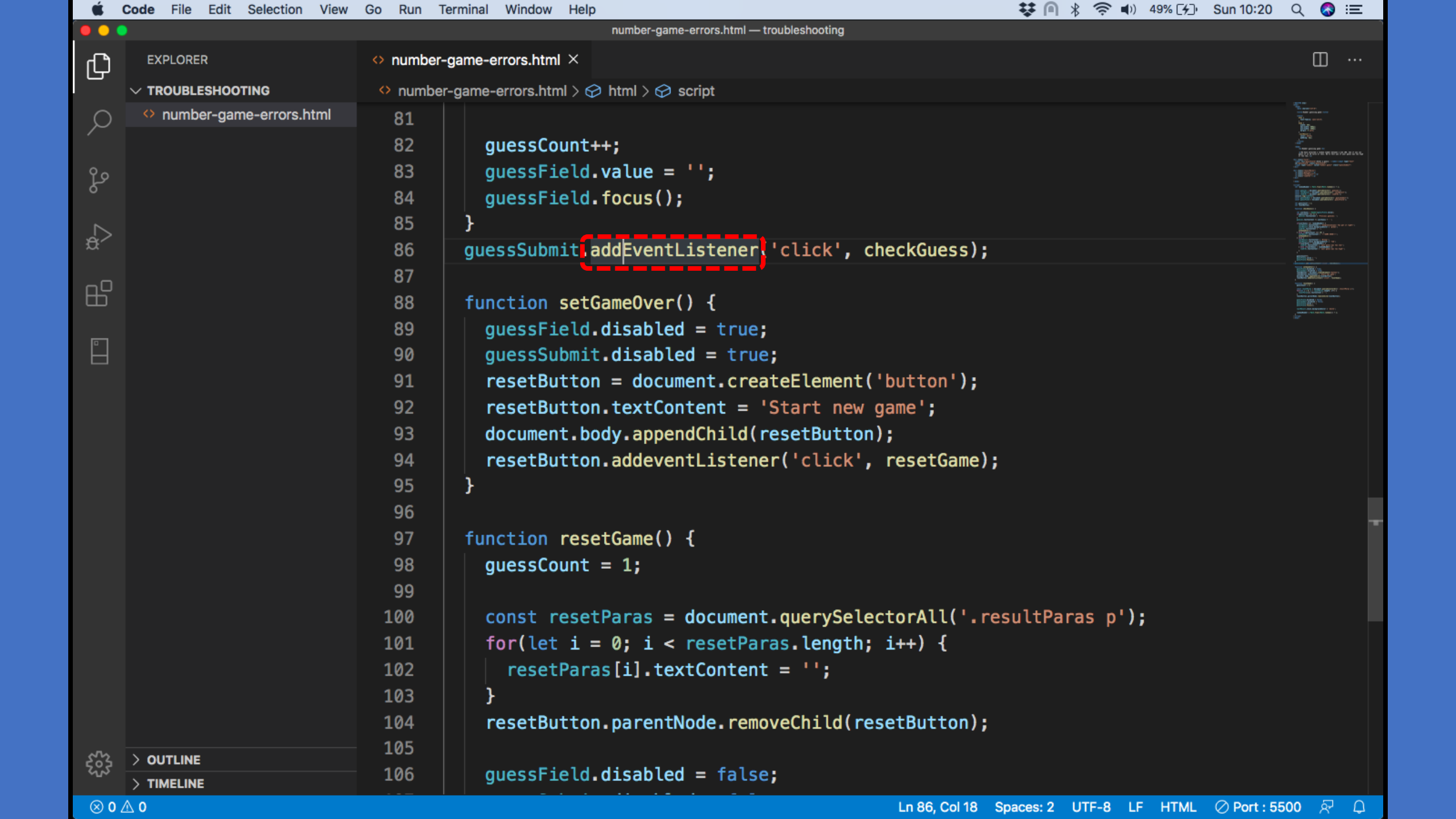
Syntax

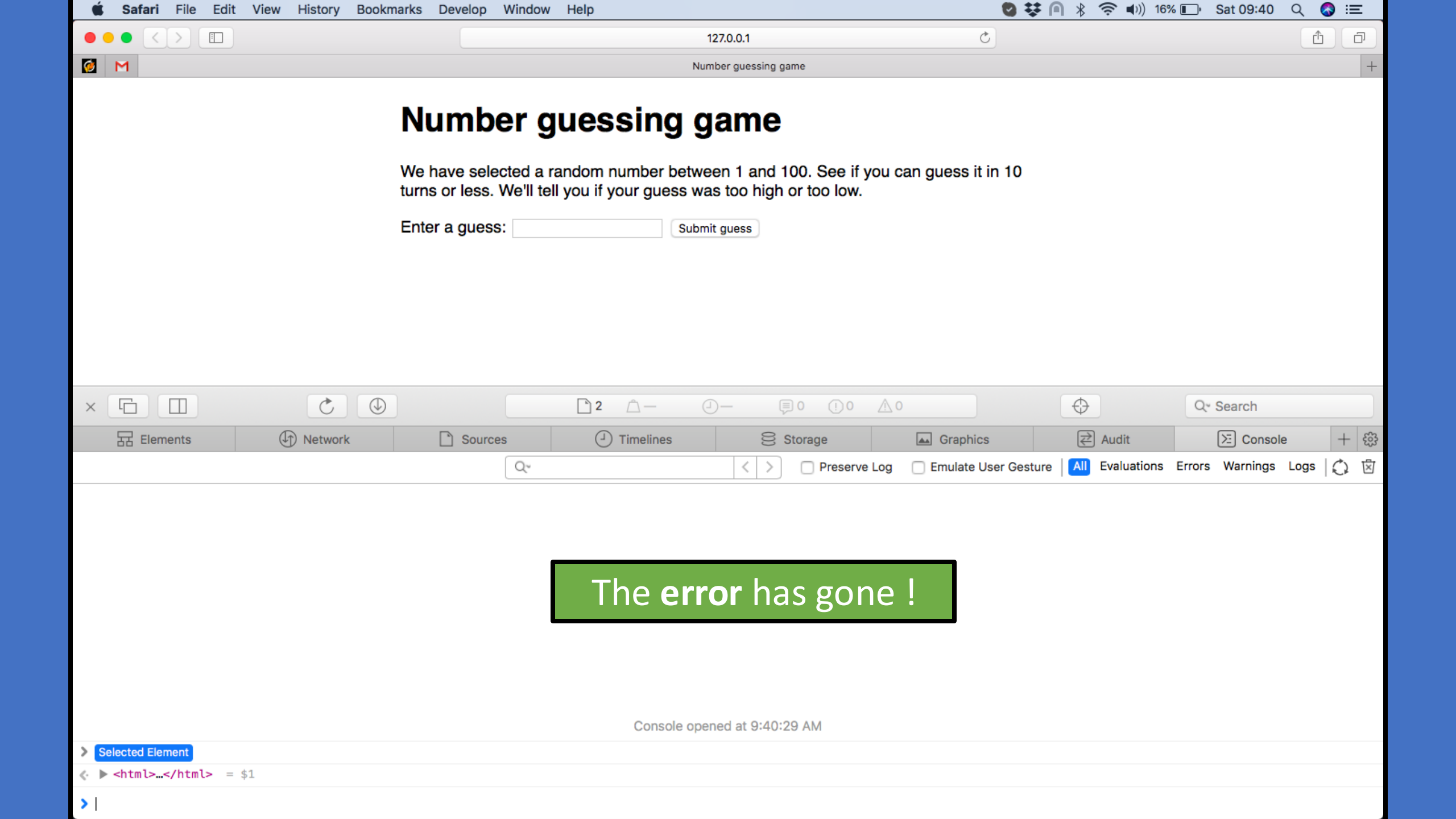
```
target.addEventListener(type, listener [, options]);
target.addEventListener(type, listener [, useCapture]);
target.addEventListener(type, listener [, useCapture, wantsUntrusted ⚠]); // Gecko/Mozilla
```

https://mzl.la/3cja0yp

Fixing syntax errors

- The **error appears** to be that we've **spelled** the **function name wrong**!
- **JavaScript** is **case sensitive**.
- Changing `addeventListener` **to** `addEventListener` **should fix** this.


Code File Edit Selection View Go Run Terminal Window Help
49% Sun 10:20
number-game-errors.html — troubleshooting
EXPLORER
TROUBLESHOOTING
number-game-errors.html
number-game-errors.html
number-game-errors.html > html > script
guessCount++;
guessField.value = '';
guessField.focus();
}
guessSubmit.addEventListener('click', checkGuess);
function setGameOver() {
guessField.disabled = true;
guessSubmit.disabled = true;
resetButton = document.createElement('button');
resetButton.textContent = 'Start new game';
document.body.appendChild(resetButton);
resetButton.addeventListener('click', resetGame);
}
function resetGame() {
guessCount = 1;
const resetParas = document.querySelectorAll('.resultParas p');
for(let i = 0; i < resetParas.length; i++) {
resetParas[i].textContent = '';
}
resetButton.parentNode.removeChild(resetButton);
guessField.disabled = false;
OUTLINE
TIMELINE
0 0
Ln 86, Col 18 Spaces: 2 UTF-8 LF HTML Port : 5500


Safari File Edit View History Bookmarks Develop Window Help
16% Sat 09:40
127.0.0.1
Number guessing game
Number guessing game
We have selected a random number between 1 and 100. See if you can guess it in 10 turns or less. We'll tell you if your guess was too high or too low.
Enter a guess:
Submit guess
Search
Elements
Network
Sources
Timelines
Storage
Graphics
Audit
Console
Preserve Log
Emulate User Gesture
All
Evaluations
Errors
Warnings
Logs
The error has gone !
Console opened at 9:40:29 AM
Selected Element
<html>…</html> = $1

Note

Note: See the MDN **TypeError: "x" is not a function** reference **page** for more **details** about this **error**.

TypeError: "x" is not a function

On this Page

- Message
- Error type
- What went wrong?
- Examples
- See also

Related Topics

JavaScript

Tutorials:

- Complete beginners
- JavaScript Guide

The JavaScript exception "is not a function" occurs when there was an attempt to call a value from a function, but the value is not actually a function.

Message

```
TypeError: Object doesn't support property or method {x} (Edge)
TypeError: "x" is not a function
```

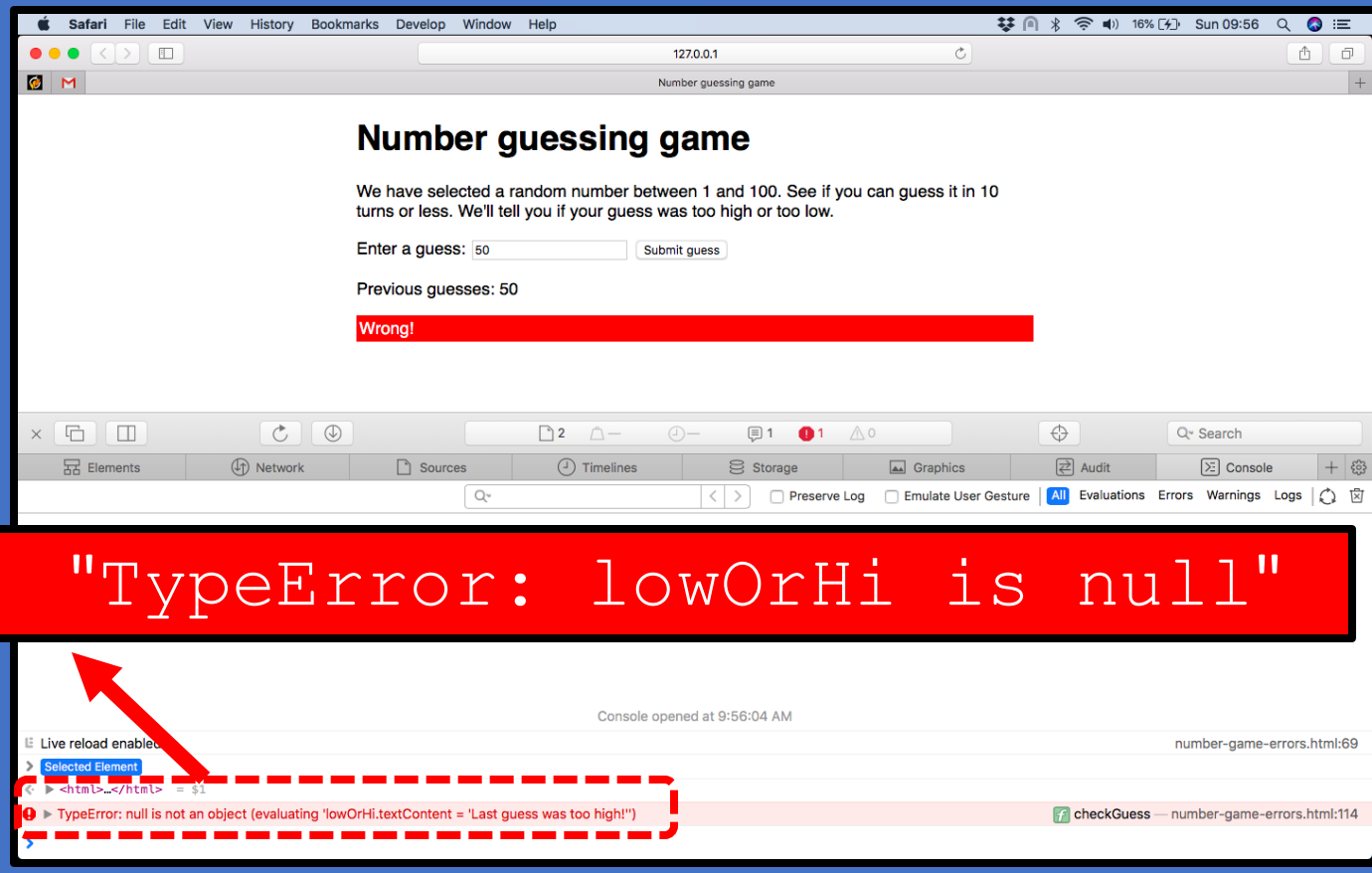
Error type

https://mzl.la/2Zi7X9l

Fixing syntax errors
(Round 2/3)

Syntax errors: round 2/3

- If you **try** to **enter** a **guess** and **press** the **Submit guess button**, you'll see **another error**!



Note

Null is a **special value** that means "**nothing**", or "**no value**". So `lowOrHi` has been **declared** and **initialized**, but **not** with any meaningful **value** — it has **no type** or **value**.

Null

Related Topics

Glossary

- JavaScript
- string
- number
- bigint
- boolean
- null
- undefined
- symbol

JavaScript data types

The JavaScript global object: `null`

Null pointer on Wikipedia

In computer science, a **`null`** value represents a reference that points, generally intentionally, to a nonexistent or invalid object or address. The meaning of a null reference varies among language implementations.

In JavaScript, `null` is marked as one of the primitive values, because its behaviour is seemingly primitive.

But in certain cases, `null` is not as "primitive" as it first seems! Every Object is derived from `null` value, and therefore `typeof` operator returns `object` for it:

```
1 | typeof null === 'object' // true
```

Last modified: Jun 29, 2020, by MDN contributors

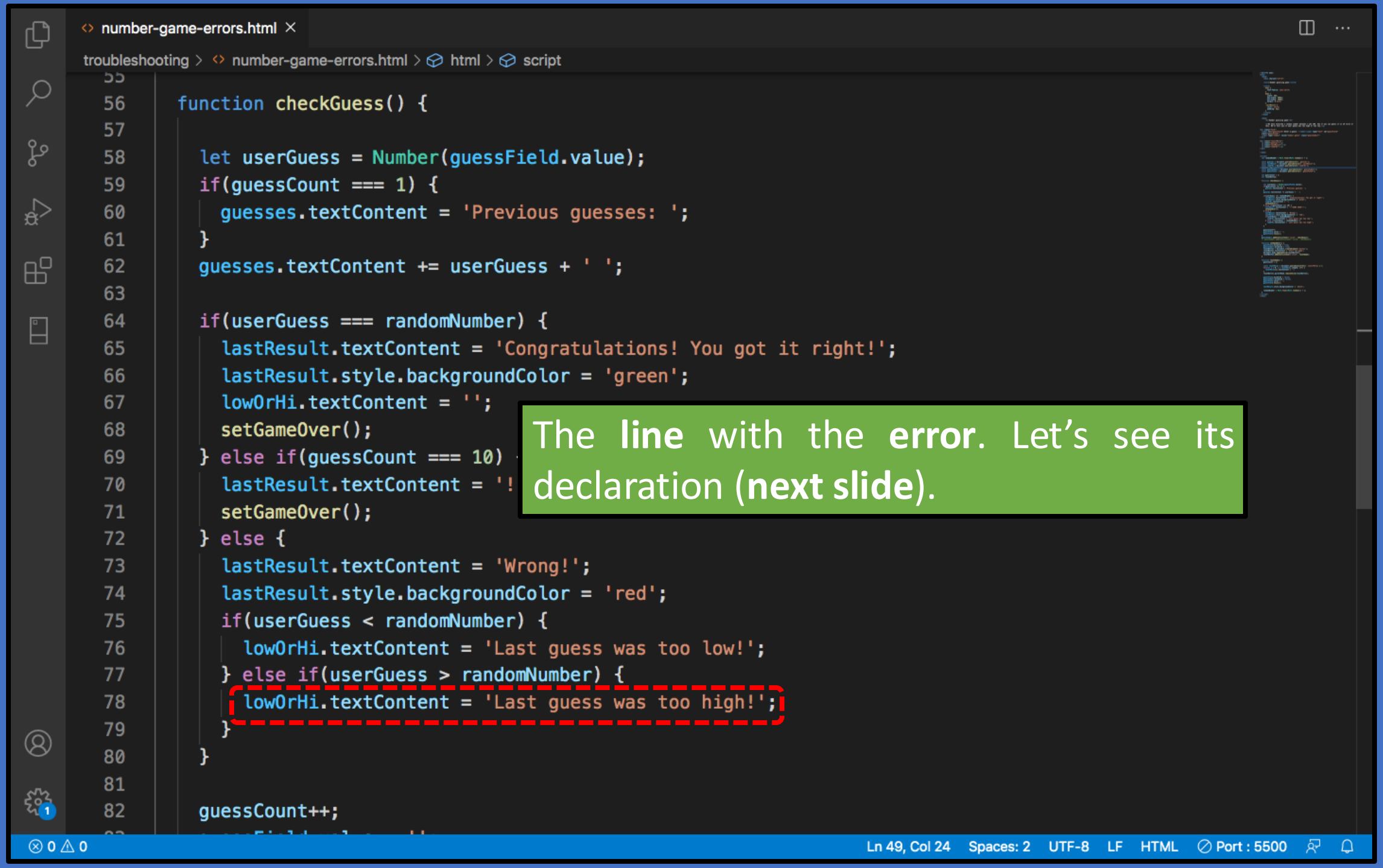
https://mzl.la/35m7irl

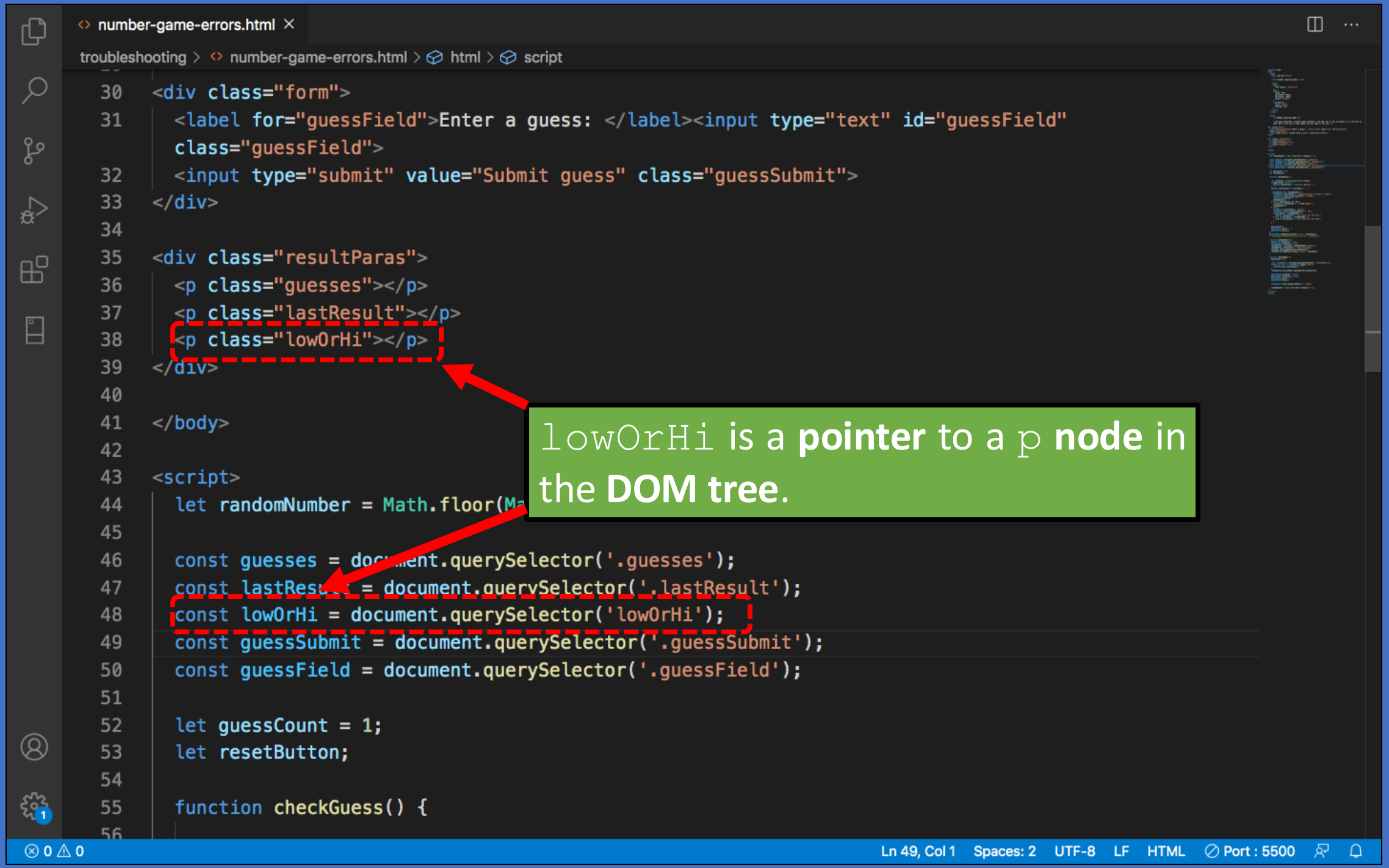
Learn the best of web development

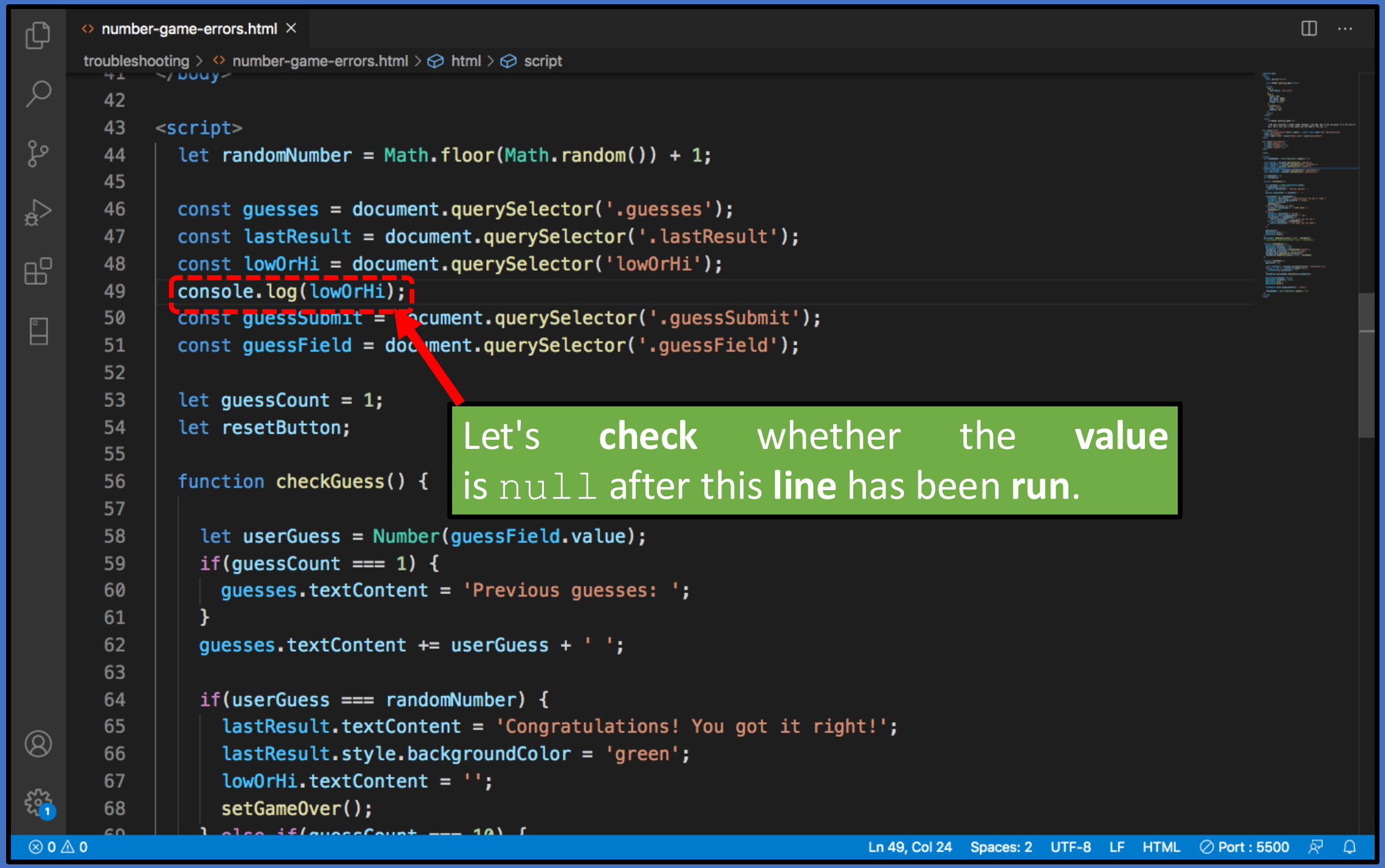
you@example.com

Note

This **error** didn't come up as **soon** as the **page** was **loaded** **because** this error **occurred** inside a **function** (inside the `checkGuess() { ... }` block). **Code inside functions** runs in a **separate scope** than **code outside functions**. In this case, the **code** was not **run** and the **error** was **not thrown until** the `checkGuess()` **function** was run.


number-game-errors.html
troubleshooting > number-game-errors.html > html > script
function checkGuess() {
let userGuess = Number(guessField.value);
if(guessCount === 1) {
guesses.textContent = 'Previous guesses: ';
}
guesses.textContent += userGuess + ' ';
if(userGuess === randomNumber) {
lastResult.textContent = 'Congratulations! You got it right!';
lastResult.style.backgroundColor = 'green';
lowOrHi.textContent = '';
setGameOver();
} else if(guessCount === 10)
lastResult.textContent = '!
setGameOver();
} else {
lastResult.textContent = 'Wrong!';
lastResult.style.backgroundColor = 'red';
if(userGuess < randomNumber) {
lowOrHi.textContent = 'Last guess was too low!';
} else if(userGuess > randomNumber) {
lowOrHi.textContent = 'Last guess was too high!';
}
}
guessCount++;
The line with the error. Let's see its declaration (next slide).
0 0
Ln 49, Col 24
Spaces: 2
UTF-8
LF
HTML
Port : 5500


number-game-errors.html
troubleshooting > number-game-errors.html > html > script
30
31 <label for="guessField">Enter a guess: </label><input type="text" id="guessField"
class="guessField">
32 <input type="submit" value="Submit guess" class="guessSubmit">
33
34
35
36
37
38
39
40
41 </body>
42
43 <script>
44 let randomNumber = Math.floor(M
45
46 const guesses = doc ment.querySelector('.guesses');
47 const lastRes = document.querySelector('.lastResult');
48 const lowOrHi = document.querySelector('lowOrHi');
49 const guessSubmit = document.querySelector('.guessSubmit');
50 const guessField = document.querySelector('.guessField');
51
52 let guessCount = 1;
53 let resetButton;
54
55 function checkGuess() {
56
lowOrHi is a pointer to a p node in the DOM tree.
0 0
Ln 49, Col 1 Spaces: 2 UTF-8 LF HTML Port : 5500


number-game-errors.html
troubleshooting > number-game-errors.html > html > script
42
43 <script>
44 let randomNumber = Math.floor(Math.random()) + 1;
45
46 const guesses = document.querySelector('.guesses');
47 const lastResult = document.querySelector('.lastResult');
48 const lowOrHi = document.querySelector('lowOrHi');
49 console.log(lowOrHi);
50 const guessSubmit = document.querySelector('.guessSubmit');
51 const guessField = document.querySelector('.guessField');
52
53 let guessCount = 1;
54 let resetButton;
55
56 function checkGuess() {
57
58 let userGuess = Number(guessField.value);
59 if(guessCount === 1) {
60 guesses.textContent = 'Previous guesses: ';
61 }
62 guesses.textContent += userGuess + ' ';
63
64 if(userGuess === randomNumber) {
65 lastResult.textContent = 'Congratulations! You got it right!';
66 lastResult.style.backgroundColor = 'green';
67 lowOrHi.textContent = '';
68 setGameOver();
Let's check whether the value is null after this line has been run.
0 0
Ln 49, Col 24
Spaces: 2
UTF-8
LF
HTML
Port : 5500

Note

`console.log()` is a **useful debugging function** that **prints** a **value** to the **console**.

console.log()

On this Page

Syntax
Difference between log() and dir()
Logging objects
Specifications
Browser compatibility
See also

Related Topics

Console API

Console

▾ Methods

assert()
clear()
count()
countReset()
debug()
dir()

The `Console` method **`log()`** outputs a message to the web console. The message may be a single string (with optional substitution values), or it may be any one or more JavaScript objects.

> **Note:** This feature is available in Web Workers.

Syntax

```
console.log(obj1 [, obj2, ..., objN]);
console.log(msg [, subst1, ..., substN]);
```

Parameters

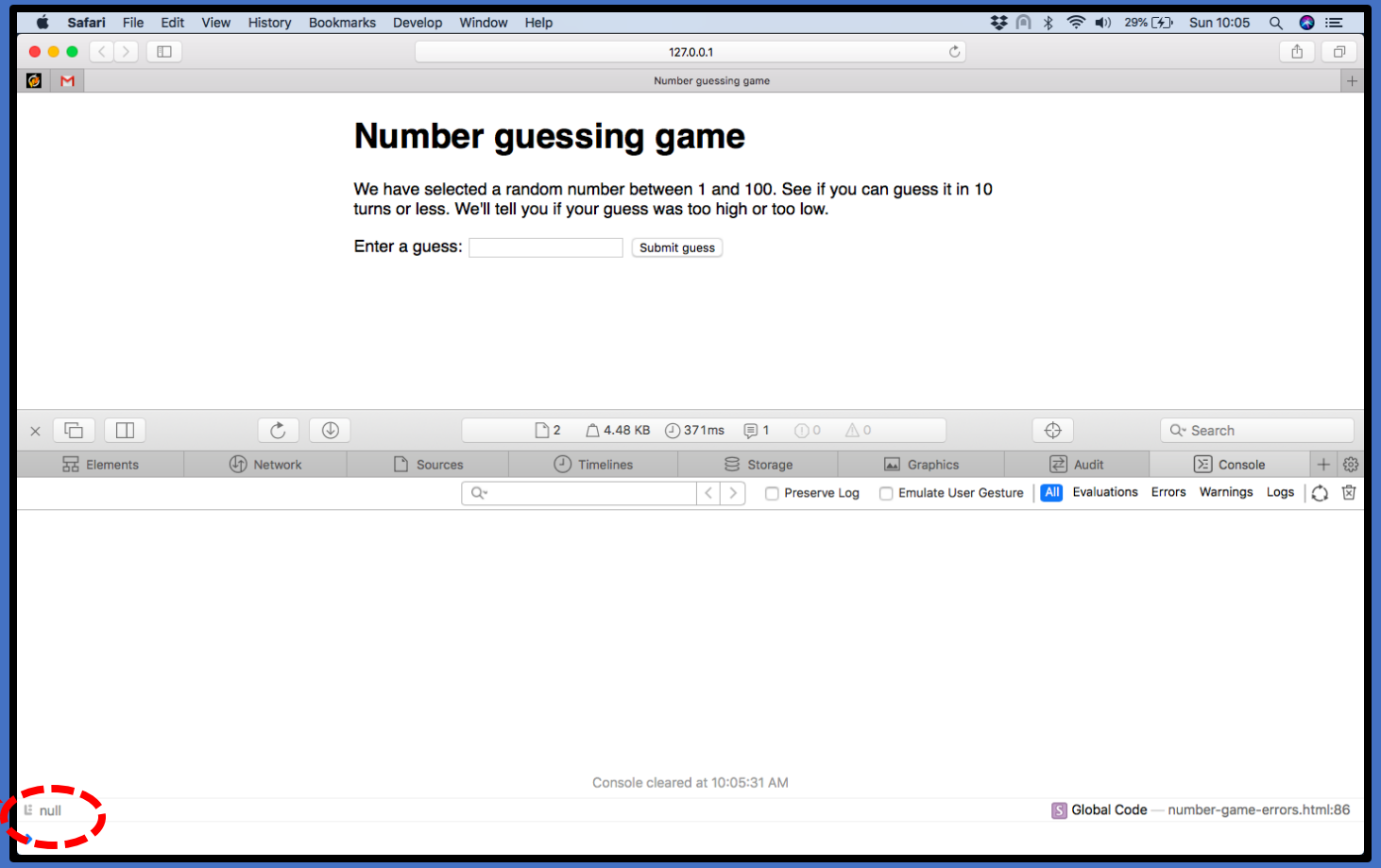
`obj1` … **`objN`**

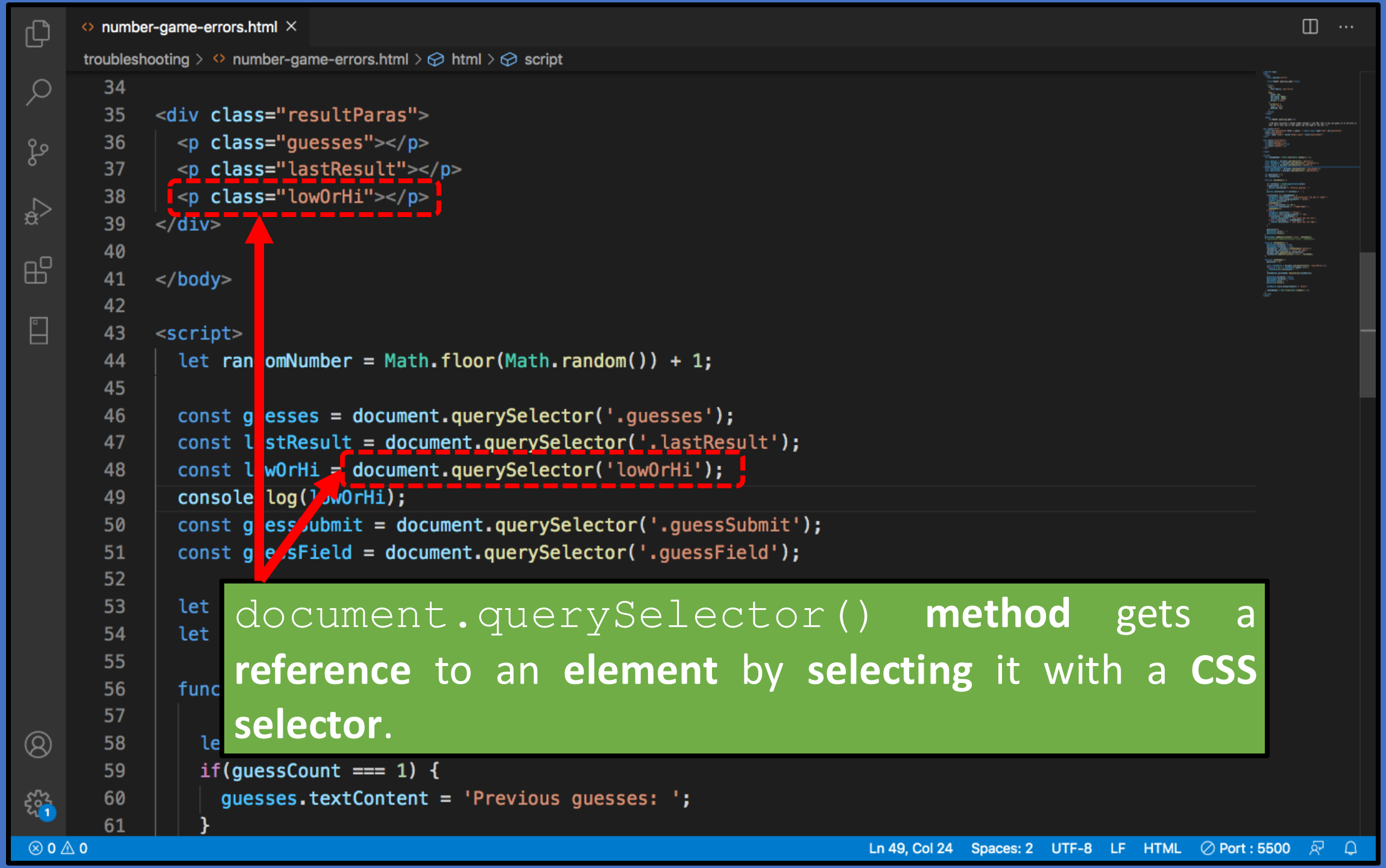
A list of JavaScript objects to output. The string representations of e
appended together in the order listed and output. Please be warned
the latest versions of Chrome and Firefox what you get logged on th
the object, which is not necessarily the 'value' of the object at the moment in time you call

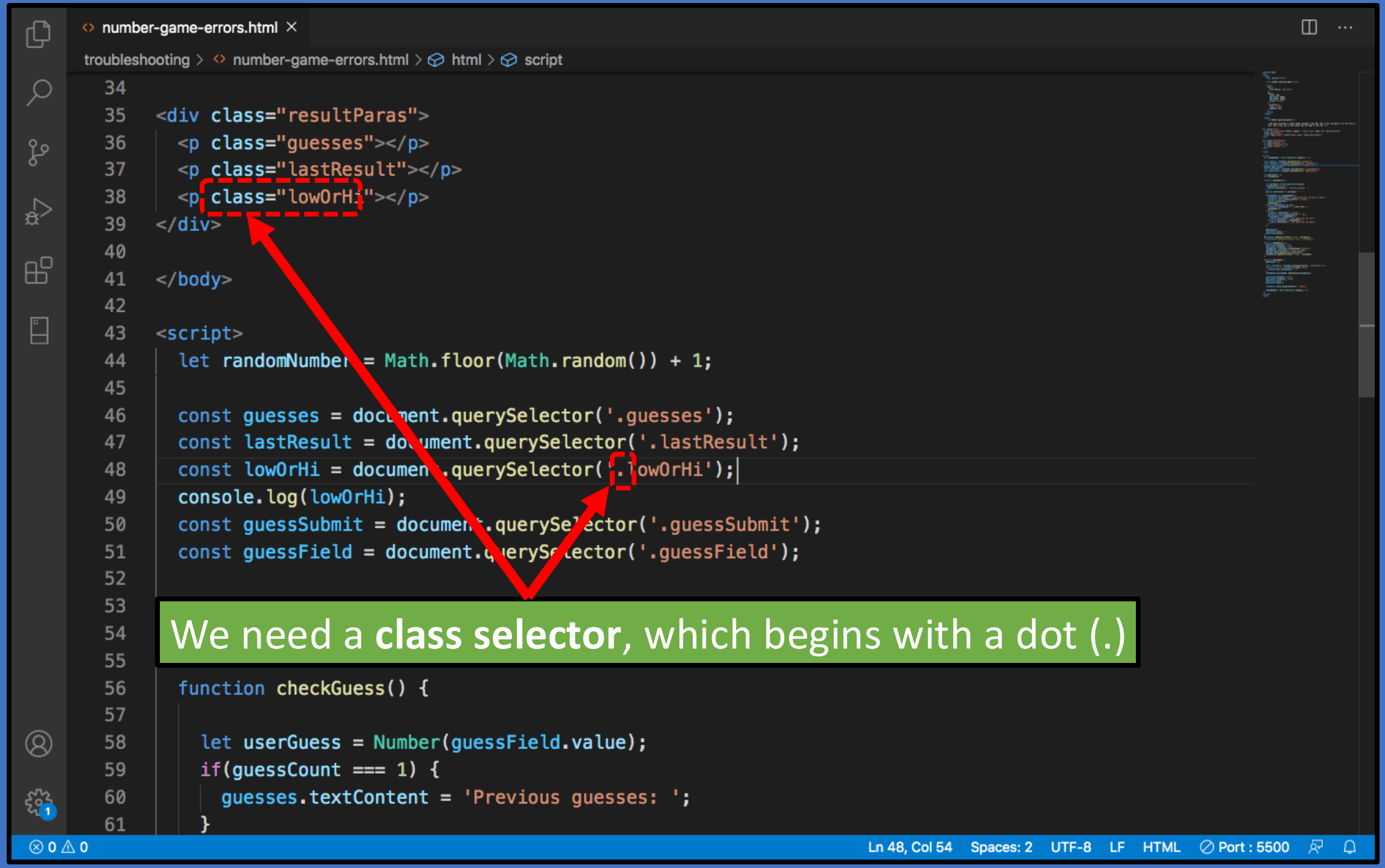
https://mzl.la/3bJAOZi

Syntax errors: round 2/3

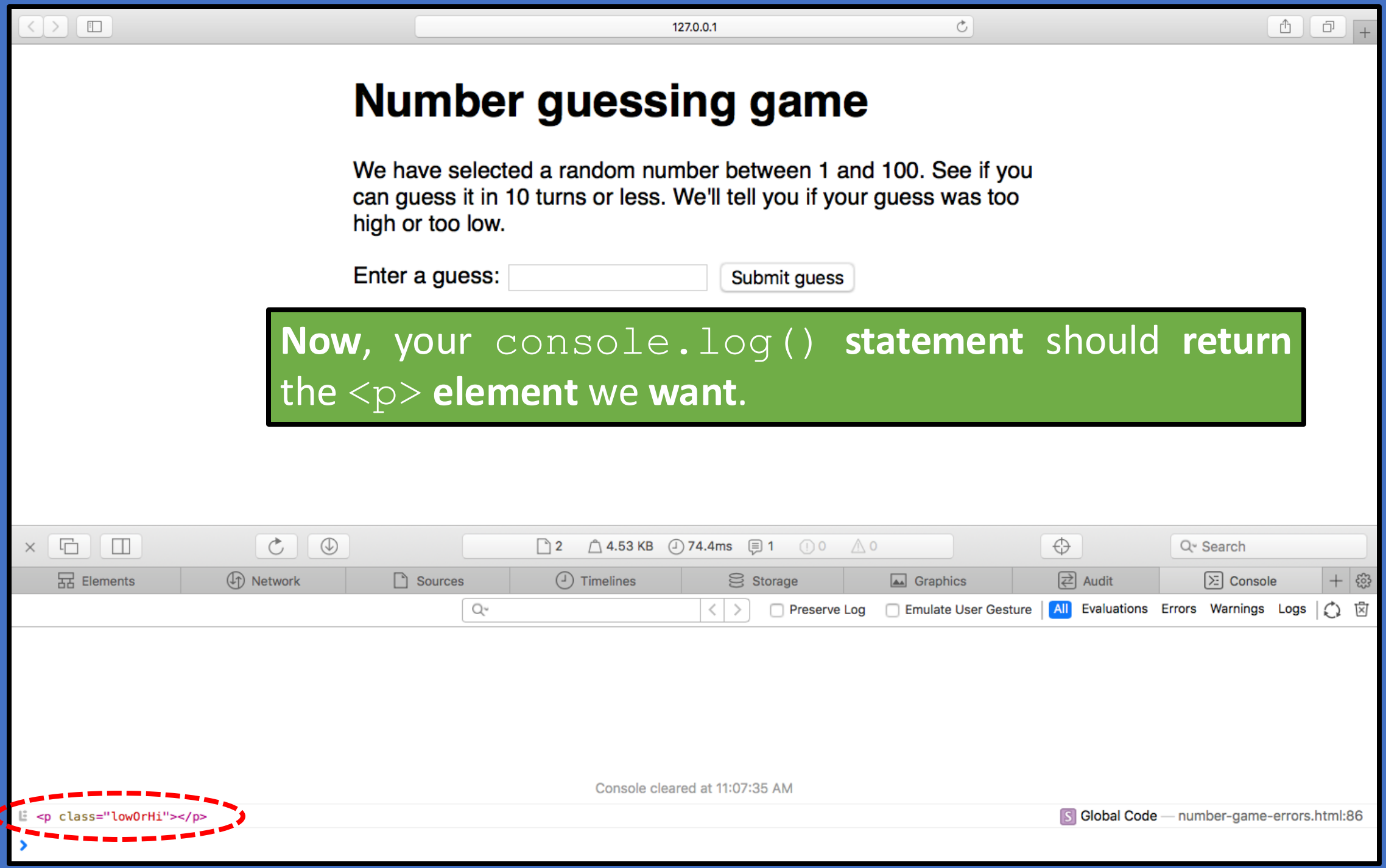
- **Save** and **refresh. You should now see** the `console.log()` **result** in your **console**.



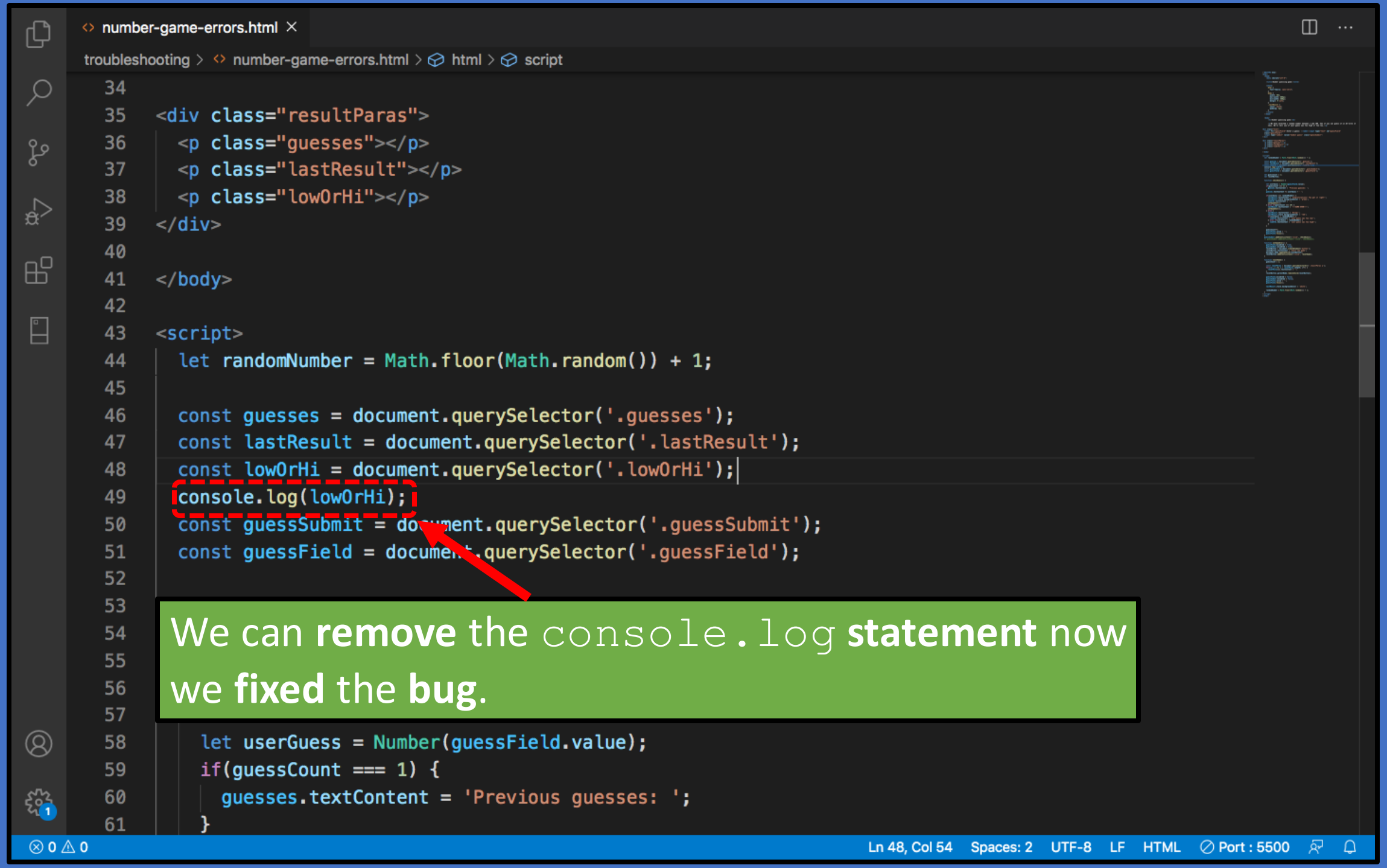

number-game-errors.html
troubleshooting > number-game-errors.html > html > script
34
35
36
37
38
39
40
41 </body>
42
43 <script>
44 let ran omNumber = Math.floor(Math.random()) + 1;
45
46 const g esses = document.querySelector('.guesses');
47 const l stResult = document.querySelector('.lastResult');
48 const l wOrHi = document.querySelector('lowOrHi');
49 console log(l wOrHi);
50 const g ess ubmit = document.querySelector('.guessSubmit');
51 const g e sField = document.querySelector('.guessField');
52
53 let
54 let
55
56 func
57
58 le
59 if(guessCount === 1) {
60 guesses.textContent = 'Previous guesses: ';
61 }
document.querySelector() method gets a reference to an element by selecting it with a CSS selector.
0 0
Ln 49, Col 24
Spaces: 2
UTF-8
LF
HTML
Port : 5500


number-game-errors.html
troubleshooting > number-game-errors.html > html > script

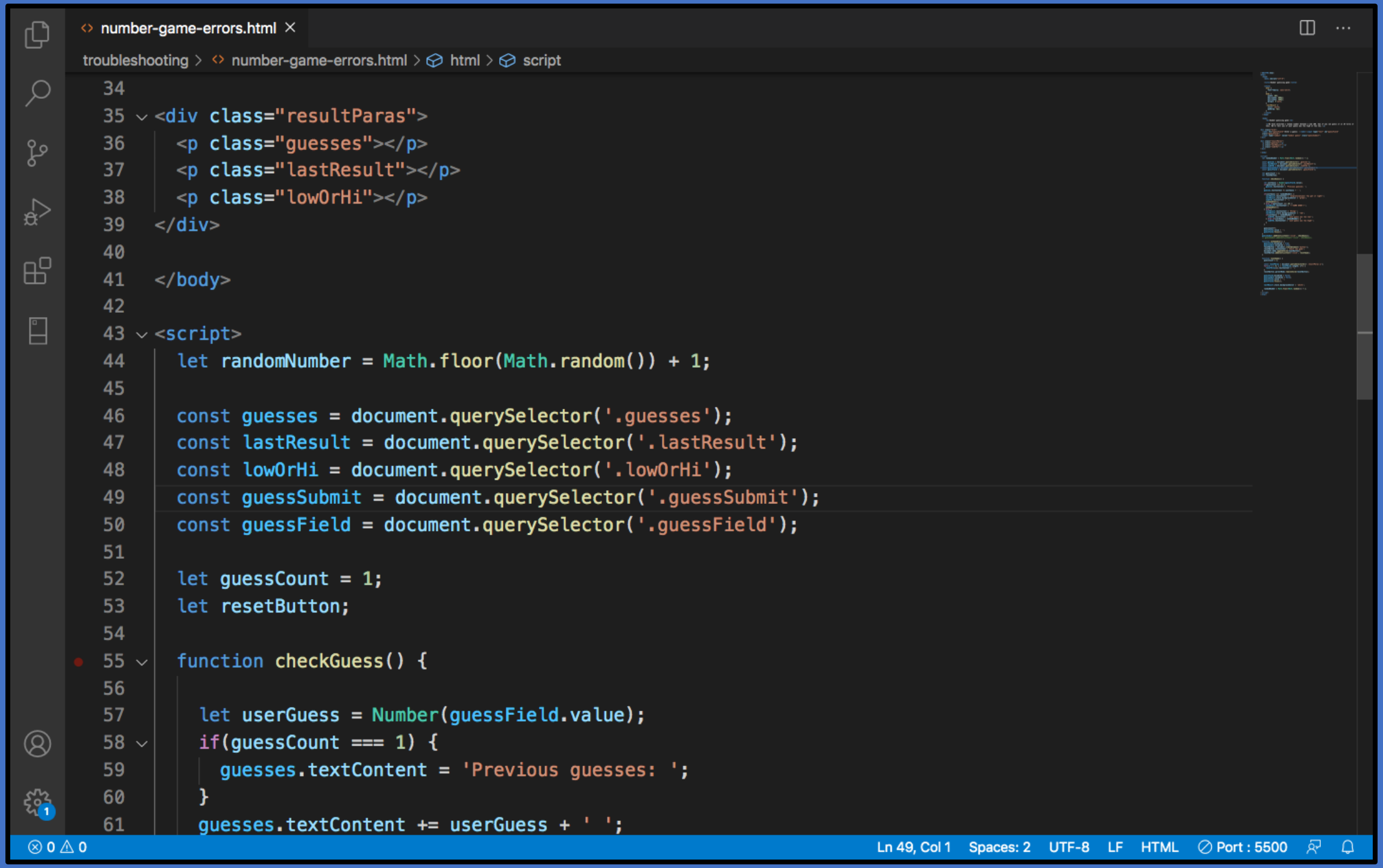
</body>
<script>
let randomNumber = Math.floor(Math.random()) + 1;
const guesses = document.querySelector('.guesses');
const lastResult = document.querySelector('.lastResult');
const lowOrHi = document.querySelector('.lowOrHi');
console.log(lowOrHi);
const guessSubmit = document.querySelector('.guessSubmit');
const guessField = document.querySelector('.guessField');
We need a class selector, which begins with a dot (.)
function checkGuess() {
let userGuess = Number(guessField.value);
if(guessCount === 1) {
guesses.textContent = 'Previous guesses: ';
}
Ln 48, Col 54
Spaces: 2
UTF-8
LF
HTML
Port : 5500


127.0.0.1
Number guessing game
We have selected a random number between 1 and 100. See if you can guess it in 10 turns or less. We'll tell you if your guess was too high or too low.
Enter a guess:
Submit guess
Now, your console.log() statement should return the element we want.
2
4.53 KB
74.4ms
1
0
0
Search
Elements
Network
Sources
Timelines
Storage
Graphics
Audit
Console
Preserve Log
Emulate User Gesture
All
Evaluations
Errors
Warnings
Logs
Console cleared at 11:07:35 AM

Global Code — number-game-errors.html:86


number-game-errors.html
troubleshooting > number-game-errors.html > html > script

</body>
<script>
let randomNumber = Math.floor(Math.random()) + 1;
const guesses = document.querySelector('.guesses');
const lastResult = document.querySelector('.lastResult');
const lowOrHi = document.querySelector('.lowOrHi');
console.log(lowOrHi);
const guessSubmit = document.querySelector('.guessSubmit');
const guessField = document.querySelector('.guessField');
We can remove the console.log statement now we fixed the bug.
let userGuess = Number(guessField.value);
if(guessCount === 1) {
guesses.textContent = 'Previous guesses: ';
}
0 0
Ln 48, Col 54
Spaces: 2
UTF-8
LF
HTML
Port : 5500

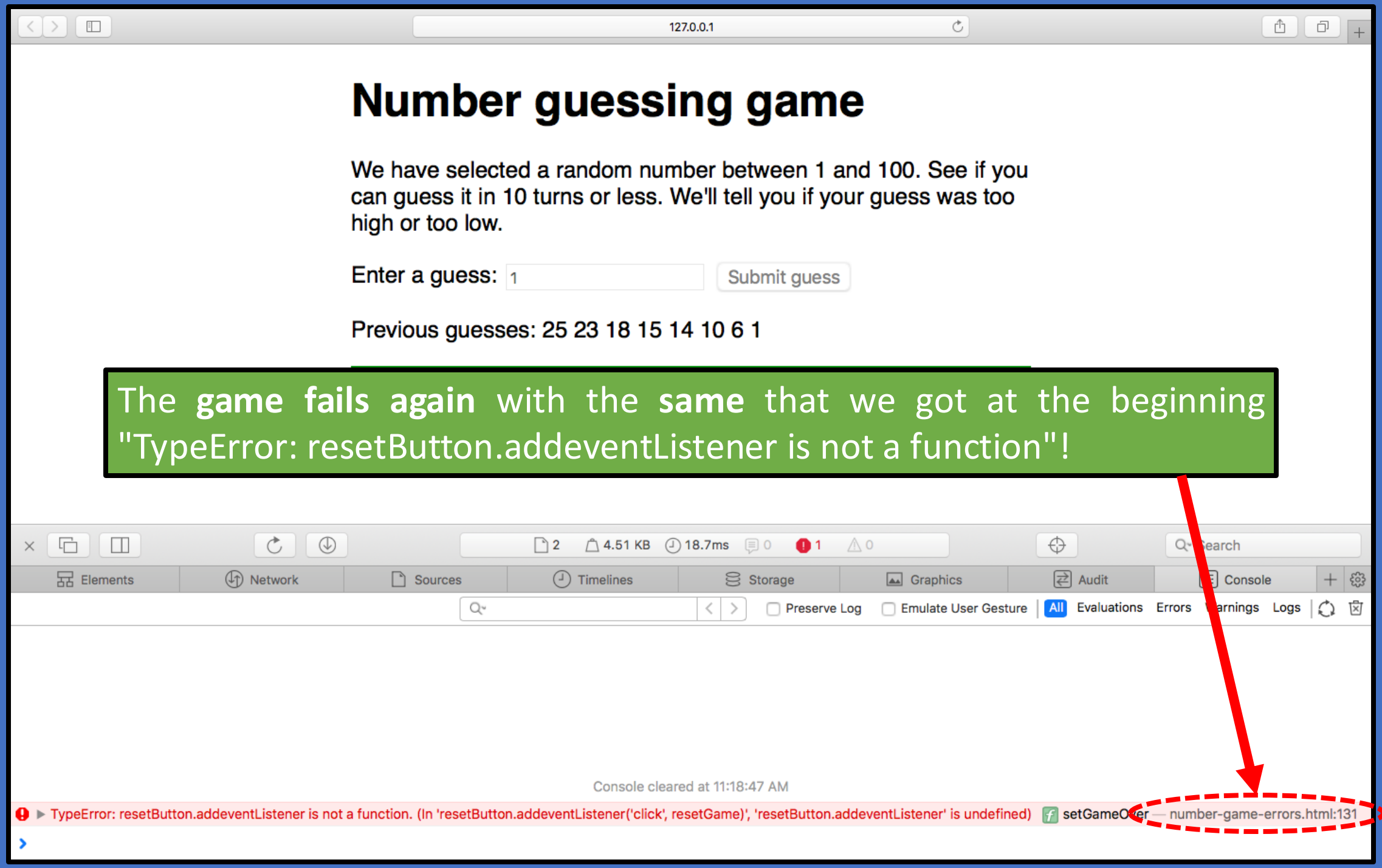

number-game-errors.html
troubleshooting > number-game-errors.html > html > script
34
35
36
37
38
39
40
41 </body>
42
43 <script>
44 let randomNumber = Math.floor(Math.random()) + 1;
45
46 const guesses = document.querySelector('.guesses');
47 const lastResult = document.querySelector('.lastResult');
48 const lowOrHi = document.querySelector('.lowOrHi');
49 const guessSubmit = document.querySelector('.guessSubmit');
50 const guessField = document.querySelector('.guessField');
51
52 let guessCount = 1;
53 let resetButton;
54
55 function checkGuess() {
56
57 let userGuess = Number(guessField.value);
58 if(guessCount === 1) {
59 guesses.textContent = 'Previous guesses: ';
60 }
61 guesses.textContent += userGuess + ' ';
0 0
Ln 49, Col 1
Spaces: 2
UTF-8
LF
HTML
Port : 5500

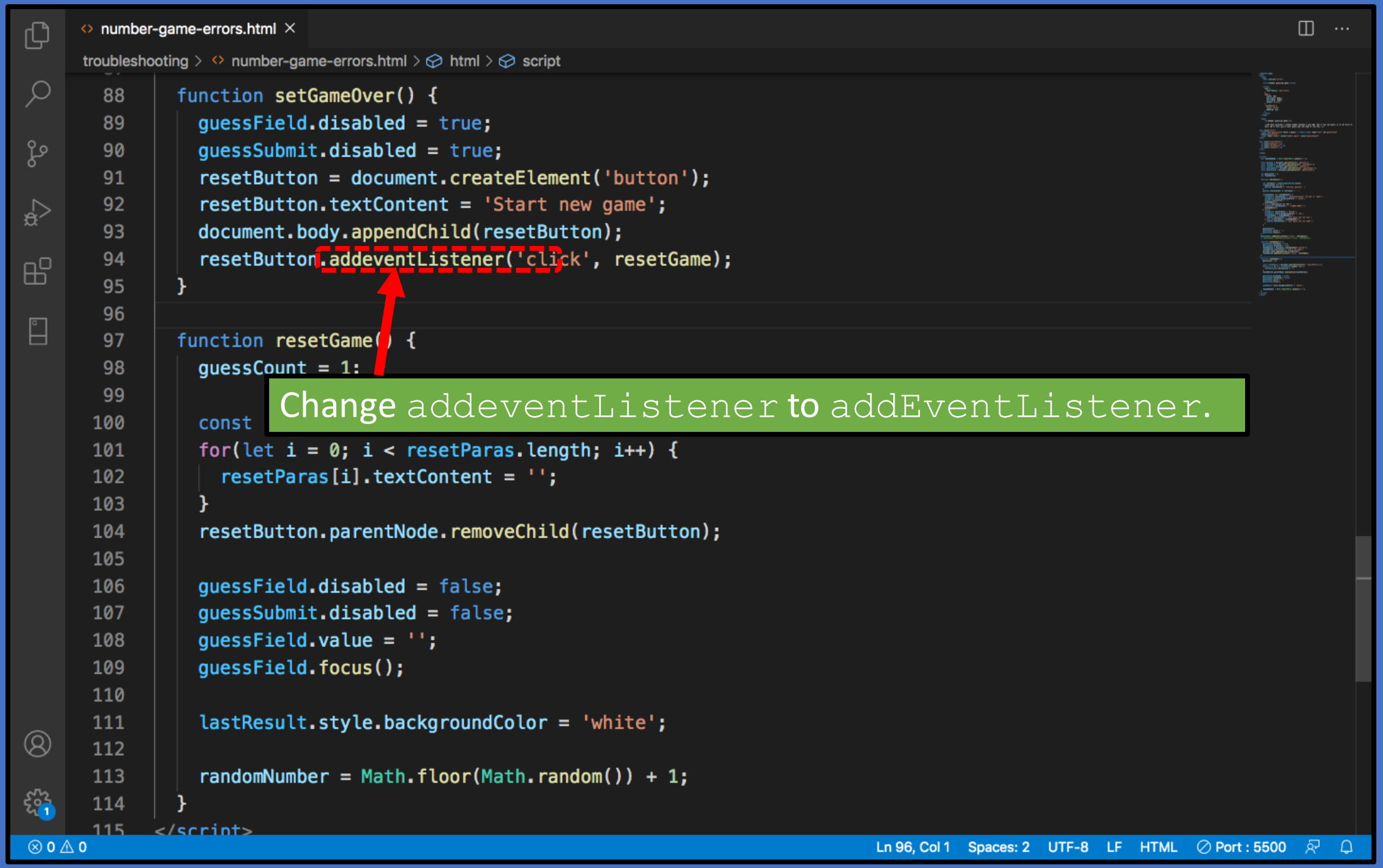
Fixing syntax errors
(Round 3/3)

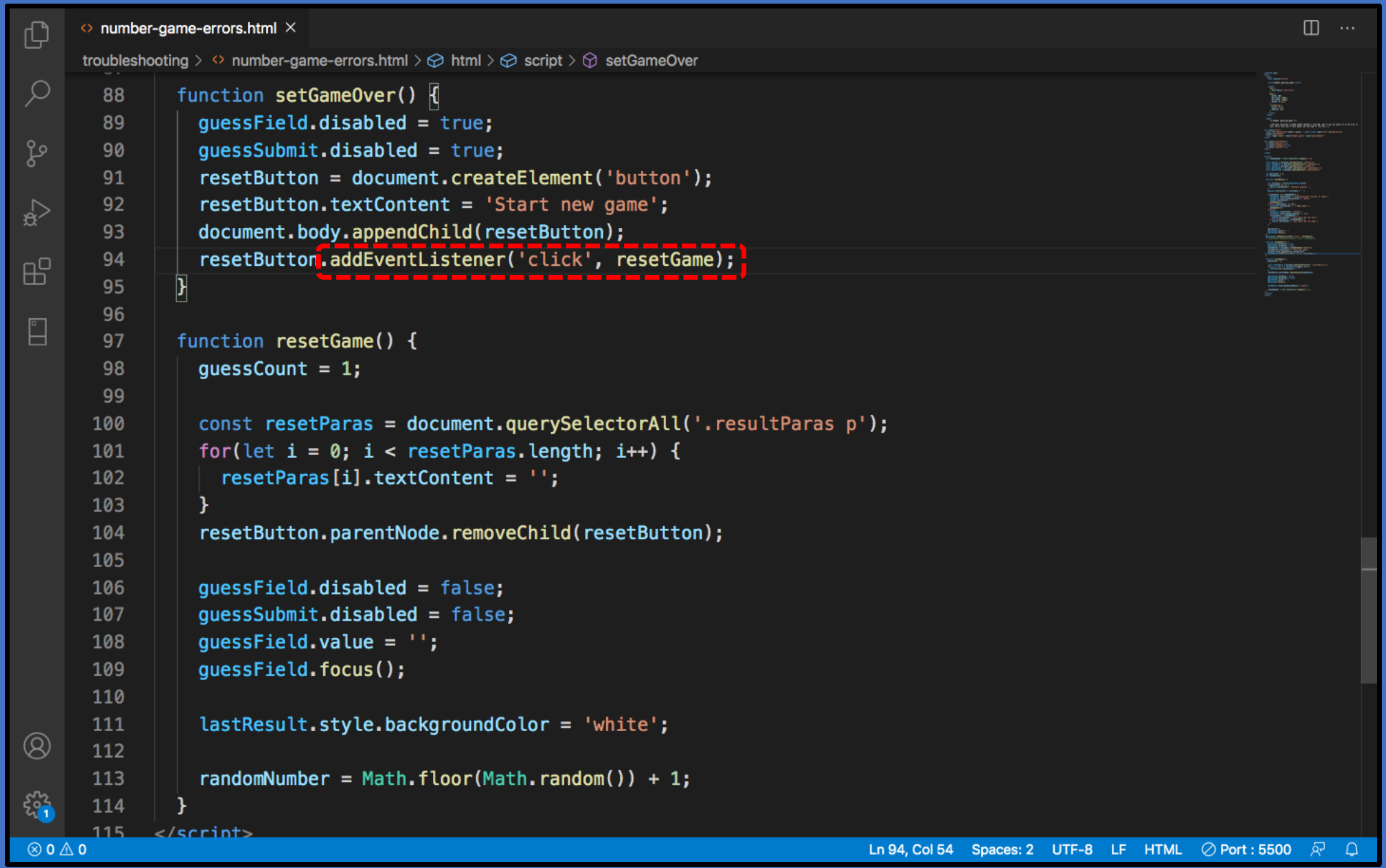
Syntax errors: round 3/3

- **Now** if you **try playing** the **game** through **again**, you should get **more success**.

- The **game** should **play fine**, **until** you **end the game**.
  - ✓**Either** by **guessing** the **right number** or by **running out** of **guesses**.


127.0.0.1
Number guessing game
We have selected a random number between 1 and 100. See if you can guess it in 10 turns or less. We'll tell you if your guess was too high or too low.
Enter a guess: 1
Submit guess
Previous guesses: 25 23 18 15 14 10 6 1
The game fails again with the same that we got at the beginning "TypeError: resetButton.addeventListener is not a function"!
2
4.51 KB
18.7ms
0
1
0
Search
Elements
Network
Sources
Timelines
Storage
Graphics
Audit
Console
Preserve Log
Emulate User Gesture
All
Evaluations
Errors
Logs
Console cleared at 11:18:47 AM
TypeError: resetButton.addeventListener is not a function. (In 'resetButton.addeventListener('click', resetGame)', 'resetButton.addeventListener' is undefined)
setGameOver
number-game-errors.html:131


number-game-errors.html
troubleshooting > number-game-errors.html > html > script
function setGameOver() {
guessField.disabled = true;
guessSubmit.disabled = true;
resetButton = document.createElement('button');
resetButton.textContent = 'Start new game';
document.body.appendChild(resetButton);
resetButton.addeventListener('click', resetGame);
}
function resetGame() {
guessCount = 1;
const
for(let i = 0; i < resetParas.length; i++) {
resetParas[i].textContent = '';
}
resetButton.parentNode.removeChild(resetButton);
guessField.disabled = false;
guessSubmit.disabled = false;
guessField.value = '';
guessField.focus();
lastResult.style.backgroundColor = 'white';
randomNumber = Math.floor(Math.random()) + 1;
}
Change addeventListener to addEventListener.
Ln 96, Col 1
Spaces: 2
UTF-8
LF
HTML
Port : 5500


number-game-errors.html
troubleshooting > number-game-errors.html > html > script > setGameOver
function setGameOver() {
guessField.disabled = true;
guessSubmit.disabled = true;
resetButton = document.createElement('button');
resetButton.textContent = 'Start new game';
document.body.appendChild(resetButton);
resetButton.addEventListener('click', resetGame);
}
function resetGame() {
guessCount = 1;
const resetParas = document.querySelectorAll('.resultParas p');
for(let i = 0; i < resetParas.length; i++) {
resetParas[i].textContent = '';
}
resetButton.parentNode.removeChild(resetButton);
guessField.disabled = false;
guessSubmit.disabled = false;
guessField.value = '';
guessField.focus();
lastResult.style.backgroundColor = 'white';
randomNumber = Math.floor(Math.random()) + 1;
}
</script>
0 0
Ln 94, Col 54
Spaces: 2
UTF-8
LF
HTML
Port : 5500

Other common syntax errors

Other common syntax errors

1. missing) after argument list
2. missing : after property id
3. missing } after function body
4. expected expression, got '*string*'
5. unterminated string literal

SyntaxError:
missing ; before statement

SyntaxError: missing ; before statement

- If we **change** this **line** inside the `checkGuess()` **function**:

```
1  var userGuess = Number(guessField.value);
```

To:

```
1  var userGuess === Number(guessField.value);
```

- It **throws** `SyntaxError: missing ; before statement`

SyntaxError: missing ; before statement

- **Don't mix** up the **assignment operator** (=) **with** the **strict equality operator** (===)
  - ✓= **sets** a **variable** to be **equal** to a **value**.
  - ✓=== **tests** whether **one value** is **equal** to **another** and **returns** a **true/false result**.

```
1  var userGuess === Number(guessField.value);
```

Note

See the MDN **SyntaxError: missing ; before statement** reference **page** for more **details** about this **error**.

SyntaxError: missing ; before statement

On this Page

- Message
- Error type
- What went wrong?
- Examples
- See also

The JavaScript exception "missing ; before statement" occurs when there is a semicolon (`;`) missing somewhere and can't be added by automatic semicolon insertion (ASI). You need to provide a semicolon, so that JavaScript can parse the source code correctly.

Message

```
SyntaxError: Expected ';' (Edge)
SyntaxError: missing ; before statement (Firefox)
```

Related Topics

JavaScript

Tutorials:

- Complete beginners
- JavaScript Guide

https://mzl.la/3hkTcZl

SyntaxError:
missing) after argument list

SyntaxError: missing) after argument list

- It **generally means** that you've **missed** the **closing parenthesis** at the **end** of a **function/method call**.

Note: See the MDN **SyntaxError: missing) after argument list** reference **page** for more **details** about this **error**.

SyntaxError: missing) after argument list

On this Page

- Message
- Error type
- What went wrong?
- Examples
- See also

Related Topics

JavaScript

Tutorials:

- Complete beginners
- JavaScript Guide

The JavaScript exception "missing) after argument list" occurs when there is an error with how a function is called. This might be a typo, a missing operator, or an unescaped string.

Message

```
SyntaxError: Expected ')' (Edge)
SyntaxError: missing ) after argument list (Firefox)
```

https://mzl.la/2FuVacm

Error type

SyntaxError:
missing : after property id

SyntaxError: missing : after property id

- This **error usually relates** to an **incorrectly formed JavaScript object**.

```
1  function checkGuess() {
```

- **To**:

```
1  function checkGuess( {
```

- This has **caused** the **browser** to **think** that we are **trying** to **pass** the **contents of the function** into the **function** as an **argument**.
  - ✓Be **careful** with those **parentheses**!

SyntaxError:
missing } after function body

SyntaxError: missing } after function body

- It **generally means** that you've **missed one of your curly braces** from a **function** or **conditional structure**.

- You can **simulate** this **error** by **deleting one** of the **closing curly braces** near the **bottom** of the `checkGuess()` **function**.

SyntaxError:
expected expression, got '*string*’

SyntaxError: expected expression, got '*string*'

- Generally **mean** that you've **left off a string value's opening or closing quote mark**.
  - ✓***string*** would be **replaced** with the **unexpected character(s)** that the **browser found** instead of a **quote mark** at the **start of a string**.

Note: See the MDN **SyntaxError: Unexpected token** reference **pages** for more **details** about these **errors**.

SyntaxError: Unexpected token

On this Page

- Message
- Error type
- What went wrong?
- Examples
- See also

The JavaScript exceptions "unexpected token" occur when a specific language construct was expected, but something else was provided. This might be a simple typo.

Message

```
SyntaxError: expected expression, got "x"
SyntaxError: expected property name, got "x"
SyntaxError: expected target, got "x"
SyntaxError: expected rest argument name, got "x"
SyntaxError: expected closing parenthesis, g
SyntaxError: expected '=>' after argument li
```

Related Topics

JavaScript

Tutorials:

- Complete beginners
- JavaScript Guide

https://mzl.la/3bX0jpV

SyntaxError:
unterminated string literal

SyntaxError: unterminated string literal

- **Means** that the **string** has not been ended with a **quote mark**.

Note: See the MDN **SyntaxError: unterminated string literal** reference **pages** for more **details** about these **errors**.

SyntaxError: unterminated string literal

On this Page

- Message
- Error type
- What went wrong?
- Examples
- See also

Related Topics

JavaScript

Tutorials:

- Complete beginners
- JavaScript Guide

The JavaScript error "unterminated string literal" occurs when there is an unterminated `String` somewhere. String literals must be enclosed by single (`'`) or double (`"`) quotes.

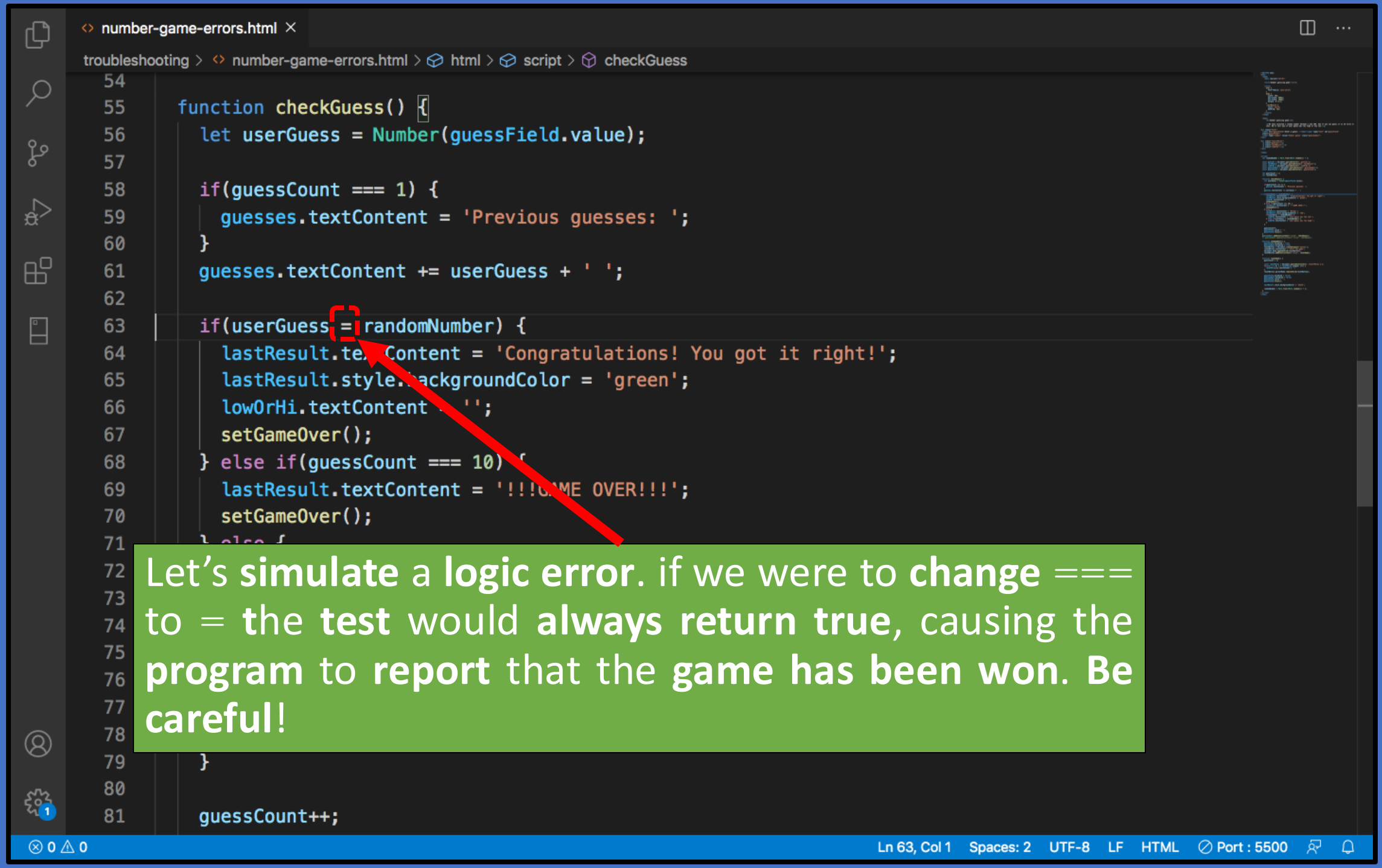
Message

```
SyntaxError: Unterminated string constant (Edge)
SyntaxError: unterminated string literal (Firefox)
```

Error type

https://mzl.la/32lQ8b8

logic errors


number-game-errors.html
troubleshooting > number-game-errors.html > html > script > checkGuess
54
55 function checkGuess() {
56 let userGuess = Number(guessField.value);
57
58 if(guessCount === 1) {
59 guesses.textContent = 'Previous guesses: ';
60 }
61 guesses.textContent += userGuess + ' ';
62
63 if(userGuess = randomNumber) {
64 lastResult.textContent = 'Congratulations! You got it right!';
65 lastResult.style.backgroundColor = 'green';
66 lowOrHi.textContent = '';
67 setGameOver();
68 } else if(guessCount === 10) {
69 lastResult.textContent = '!!!GAME OVER!!!';
70 setGameOver();
71
72 Let's simulate a logic error. if we were to change === to = the test would always return true, causing the program to report that the game has been won. Be careful!
79 }
80
81 guessCount++;
⊗ 0 ⚠ 0
Ln 63, Col 1
Spaces: 2
UTF-8
LF
HTML
Port : 5500

127.0.0.1

Number guessing game

We have selected a random number between
can guess it in 10 turns or less. We'll tell you i
high or too low.

Enter a guess: Submit gu

Previous guesses: 34

Congratulations! You got it right!

127.0.0.1

Number guessing game

We have selected a random number between 1 and 100. See if you can guess it in 10 turns or less. We'll tell you if your guess was too high or too low.

Enter a guess: Submit guess

Previous guesses: 56

Congratulations! You got it right!

Start new game

127.0.0.1

Number guessing game

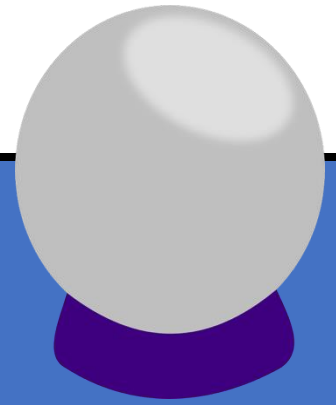
We have selected a random number between 1 and 100. See if you can guess it in 10 turns or less. We'll tell you if your guess was too high or too low.

Enter a guess: Submit guess

Previous guesses: 99

Congratulations! You got it right!

Start new game



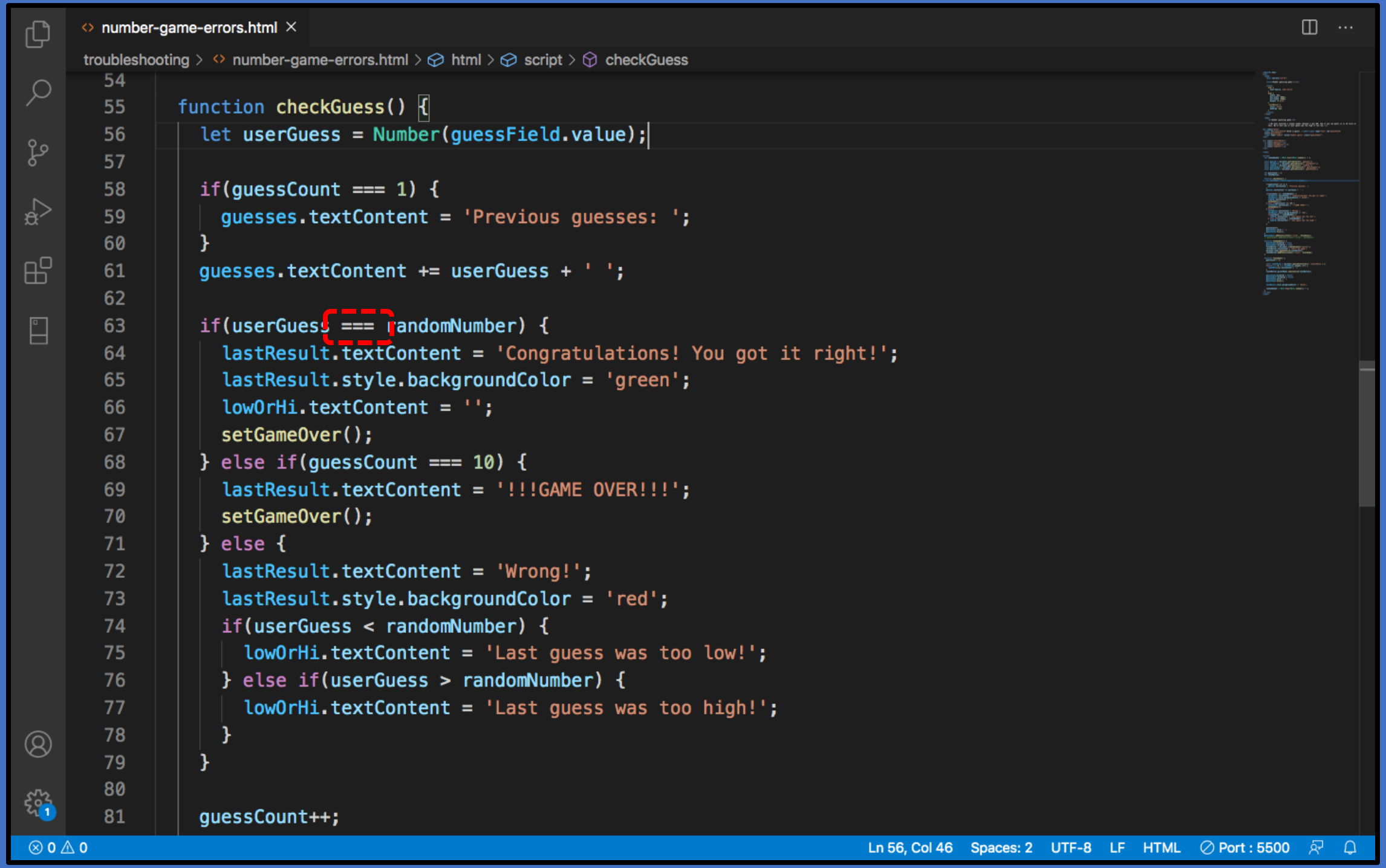
Source: https://bit.ly/33kPlXm

Do you always win?

- The **program always says** you've **won**, **regardless** of the **guess** you **enter**.
- It could be a **symptom** of **mixing up** the **assignment** and **strict equality operators**.



Source: https://bit.ly/2GU0BT6


number-game-errors.html
troubleshooting > number-game-errors.html > html > script > checkGuess
54
55 function checkGuess() {
56 let userGuess = Number(guessField.value);
57
58 if(guessCount === 1) {
59 guesses.textContent = 'Previous guesses: ';
60 }
61 guesses.textContent += userGuess + ' ';
62
63 if(userGuess === randomNumber) {
64 lastResult.textContent = 'Congratulations! You got it right!';
65 lastResult.style.backgroundColor = 'green';
66 lowOrHi.textContent = '';
67 setGameOver();
68 } else if(guessCount === 10) {
69 lastResult.textContent = '!!!GAME OVER!!!';
70 setGameOver();
71 } else {
72 lastResult.textContent = 'Wrong!';
73 lastResult.style.backgroundColor = 'red';
74 if(userGuess < randomNumber) {
75 lowOrHi.textContent = 'Last guess was too low!';
76 } else if(userGuess > randomNumber) {
77 lowOrHi.textContent = 'Last guess was too high!';
78 }
79 }
80
81 guessCount++;
0 0
Ln 56, Col 46
Spaces: 2
UTF-8
LF
HTML
Port : 5500

127.0.0.1

Number guessing game

We have selected a random number between 1 and 100
can guess it in 10 turns or less. We'll tell you if your gue
high or too low.

Enter a guess: Submit guess

Previous guesses: 33

Wrong!

Last guess was too high!

127.0.0.1

Number guessing game

We have selected a random number between 1 and 100. See if you can guess it in 10 turns or less. We'll tell you if your guess was too high or too low.

Enter a guess: Submit guess

Previous guesses: 99

Wrong!

Last guess was too high!

127.0.0.1

Number guessing game

We have selected a random number between 1 and 100. See if you can guess it in 10 turns or less. We'll tell you if your guess was too high or too low.

Enter a guess: Submit guess

Previous guesses: 85

Wrong!

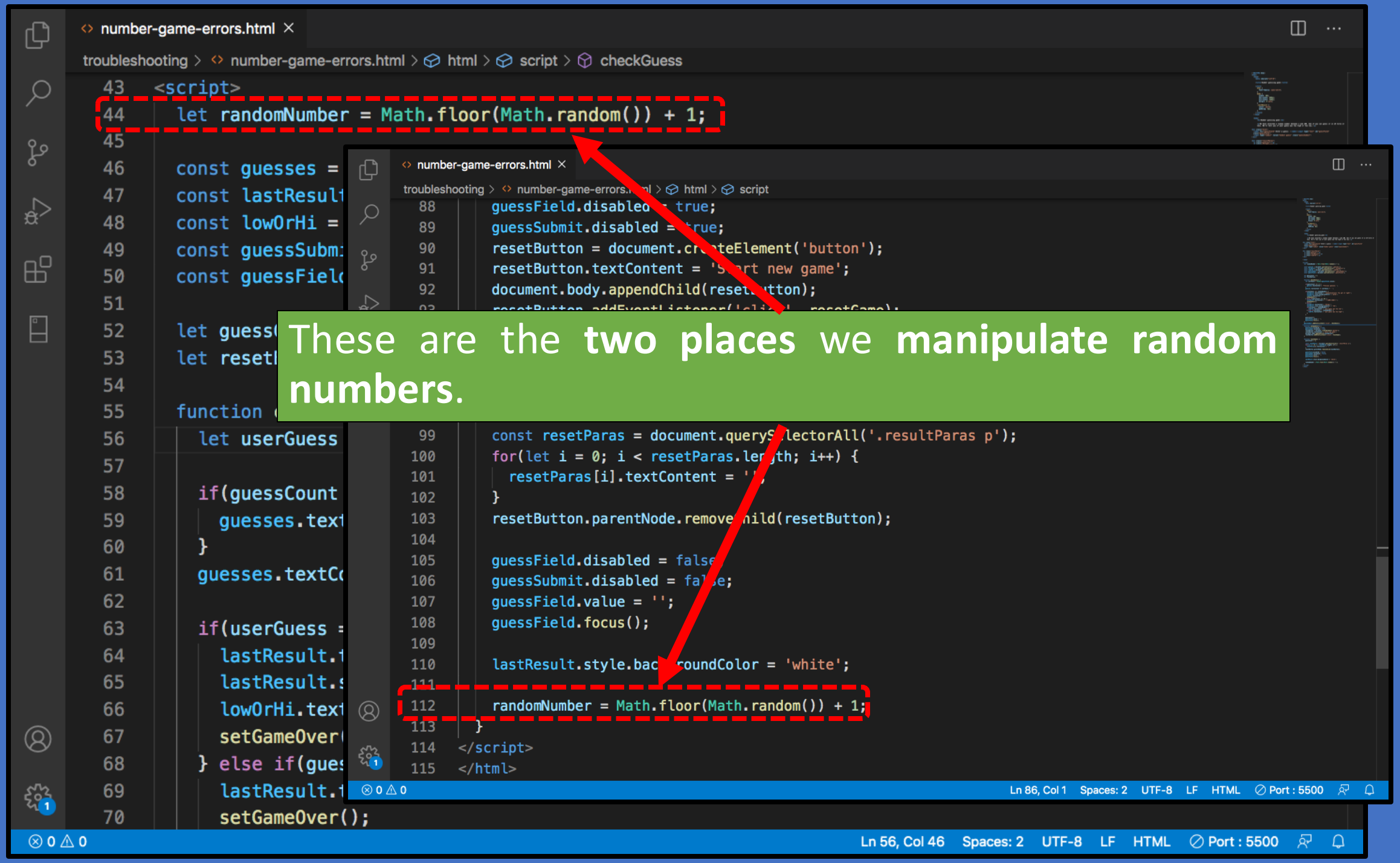
Last guess was too high!

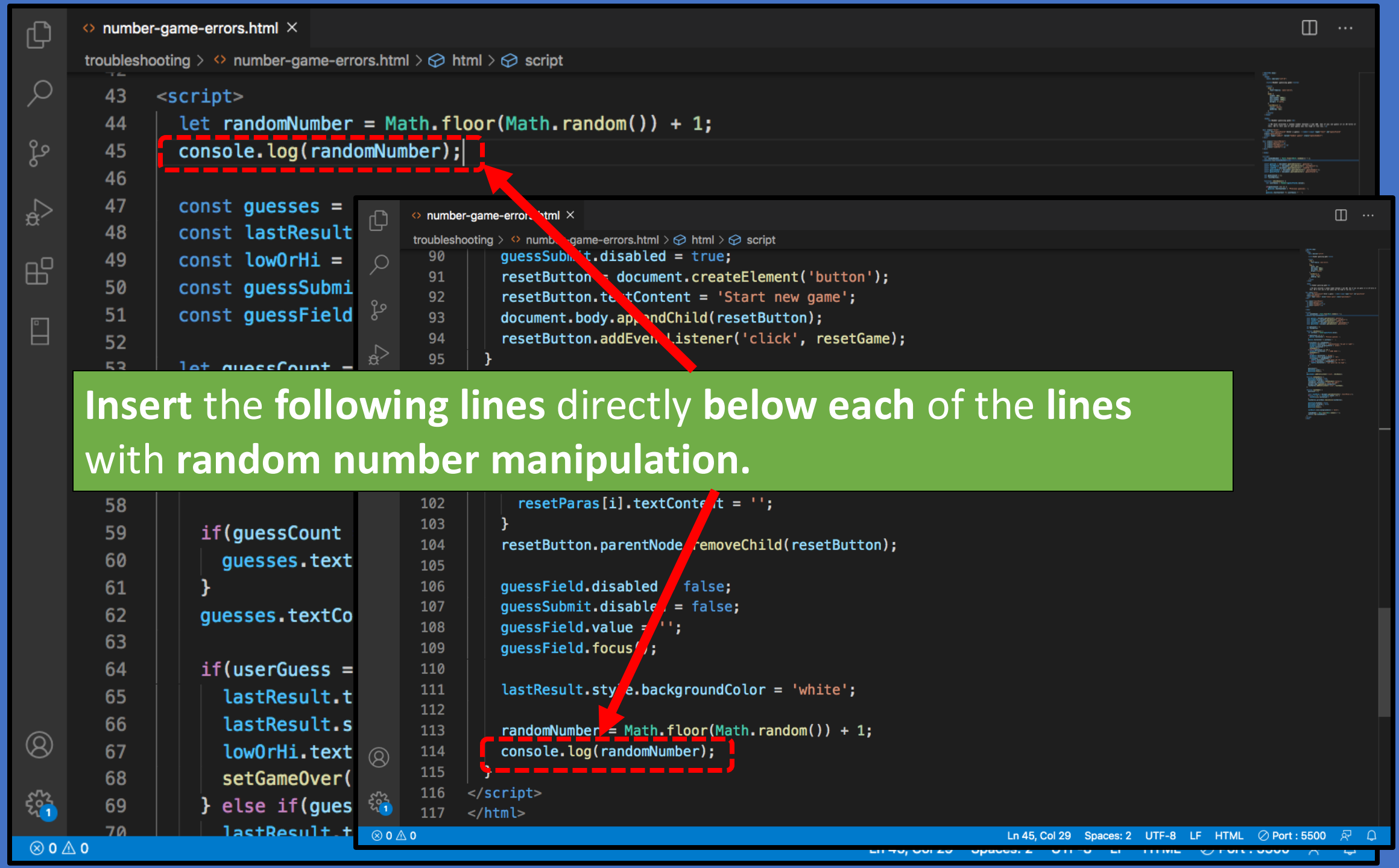


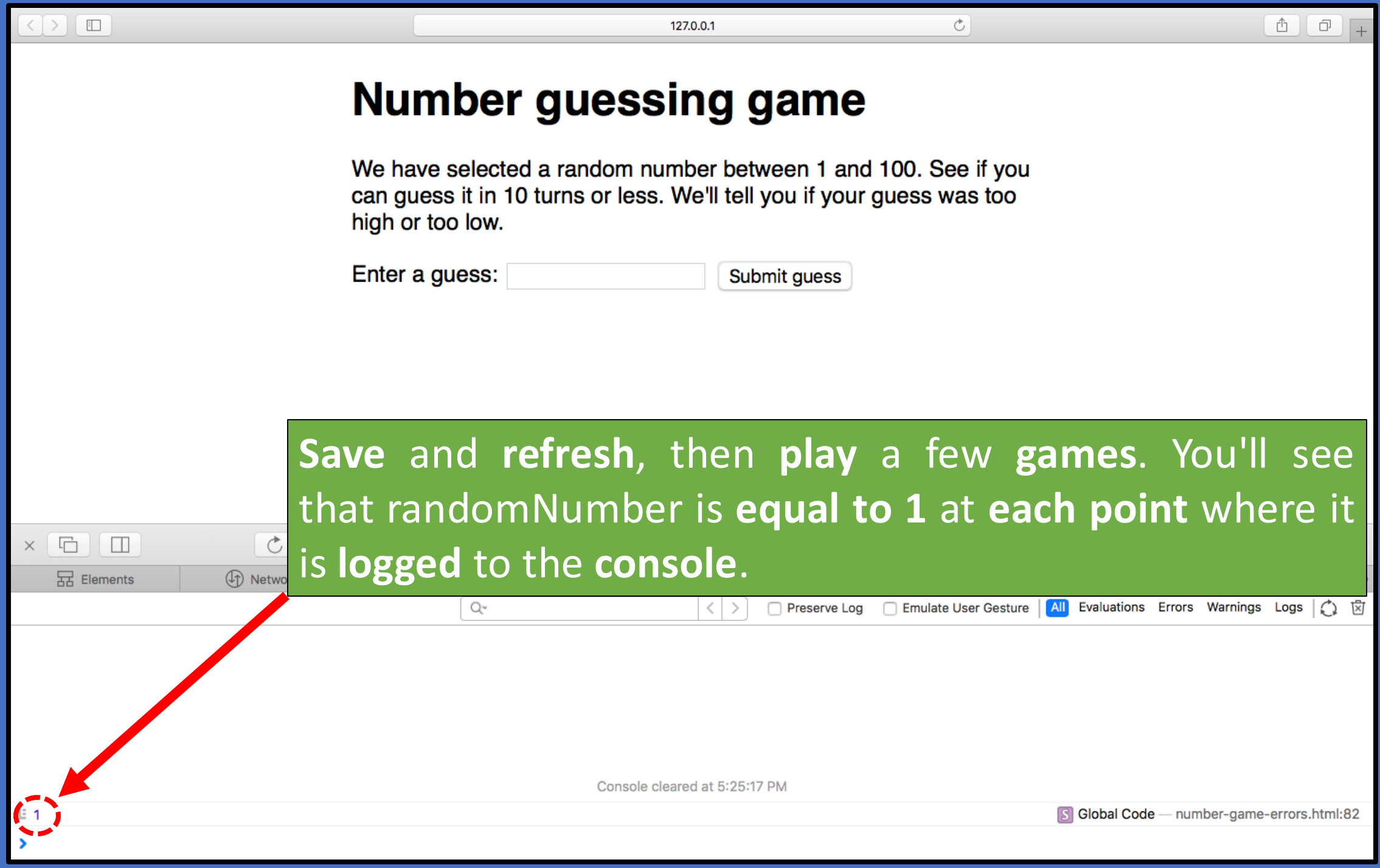
Random number always equals to 1?

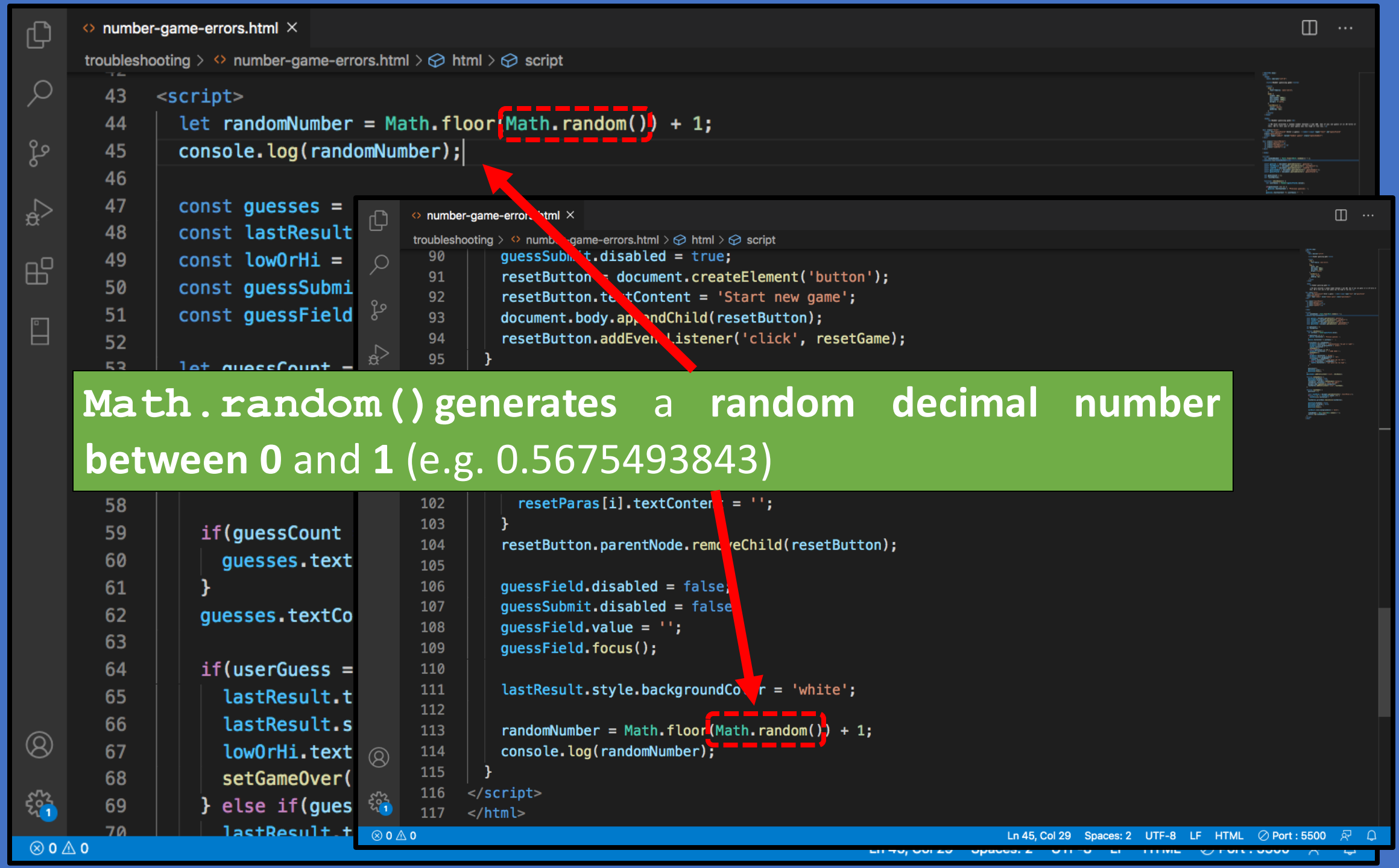
- After **playing** through a **few times** you'll **notice** that the "**random**" number you've got to guess is **always 1**.

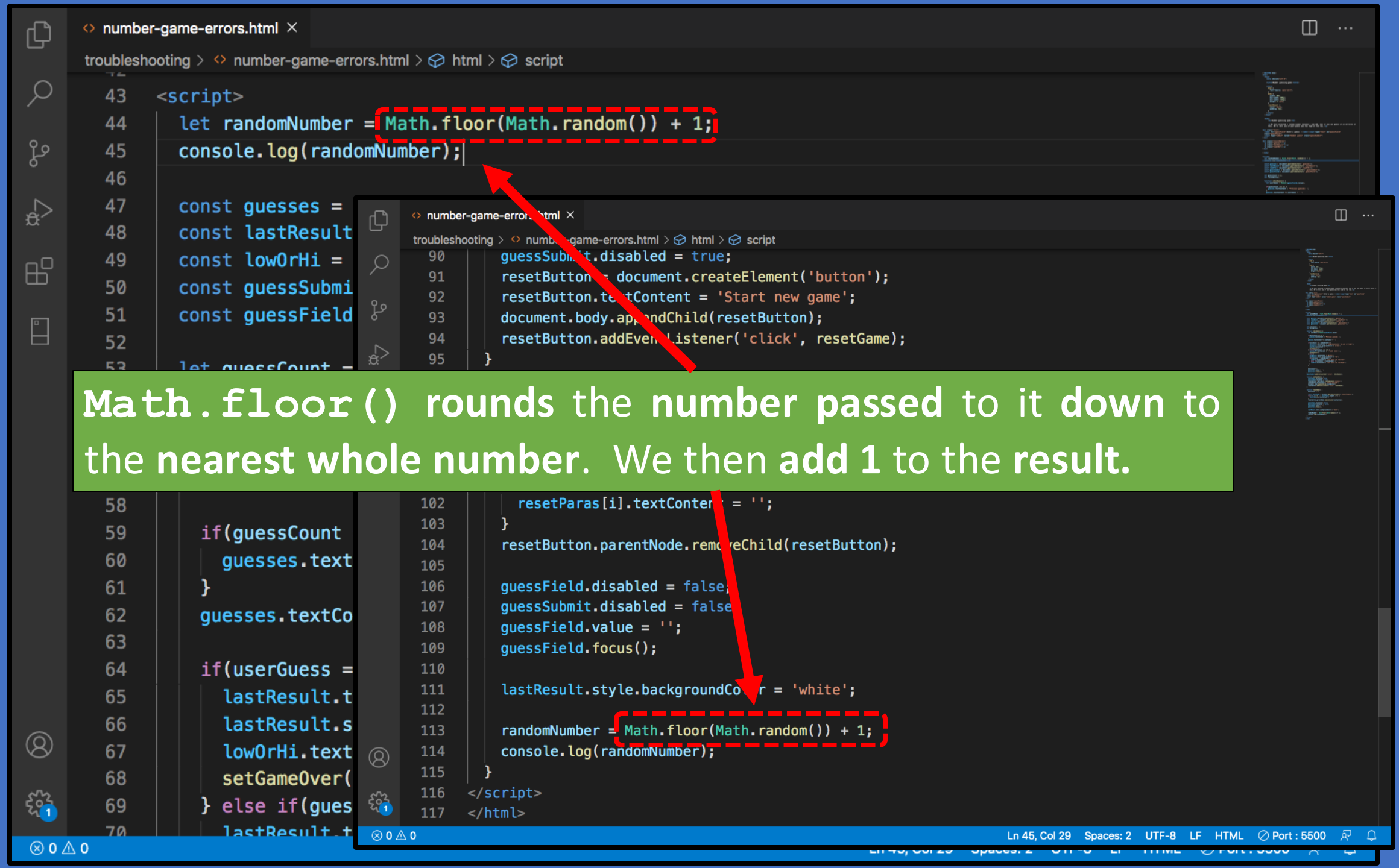
- There's a **problem** in the **game logic** somewhere
  - ✓The **game** is **not returning** an **error**;
    - It just **isn't playing right**.

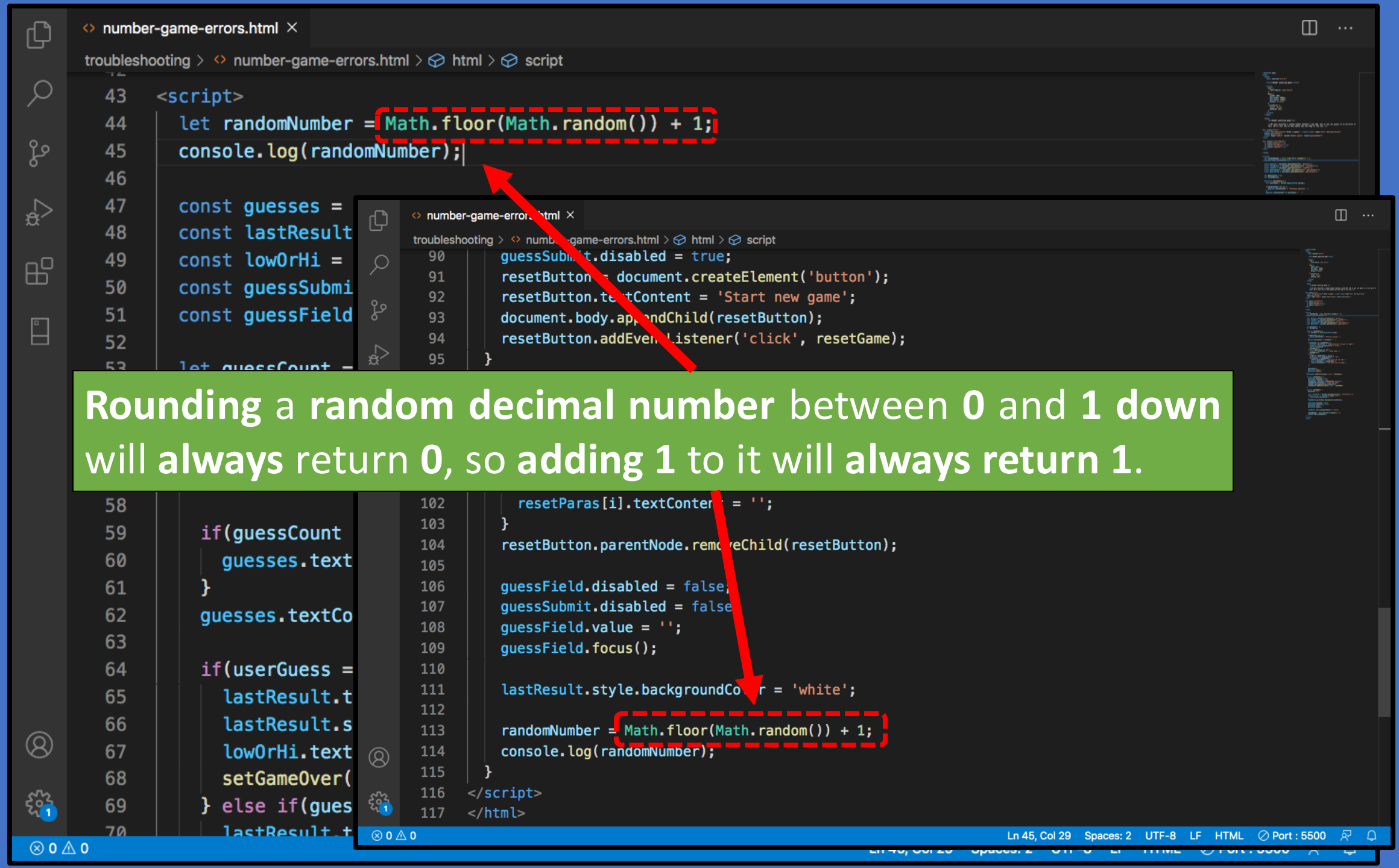

number-game-errors.html
troubleshooting > number-game-errors.html > html > script > checkGuess
<script>
let randomNumber = Math.floor(Math.random()) + 1;
const guesses =
const lastResult
const lowOrHi =
const guessSubm
const guessField
let guess
let reset
function
let userGuess
if(guessCount
guesses.text
}
guesses.textC
if(userGuess
lastResult.t
lastResult.s
lowOrHi.text
setGameOver
} else if(gues
lastResult.t
setGameOver();
number-game-errors.html
troubleshooting > number-game-errors.html > html > script
guessField.disabled = true;
guessSubmit.disabled = true;
resetButton = document.createElement('button');
resetButton.textContent = 'Start new game';
document.body.appendChild(resetButton);
const resetParas = document.querySelectorAll('.resultParas p');
for(let i = 0; i < resetParas.length; i++) {
resetParas[i].textContent = '';
}
resetButton.parentNode.removeChild(resetButton);
guessField.disabled = false;
guessSubmit.disabled = false;
guessField.value = '';
guessField.focus();
lastResult.style.backgroundColor = 'white';
randomNumber = Math.floor(Math.random()) + 1;
}
</script>
</html>
These are the two places we manipulate random numbers.
Ln 86, Col 1 Spaces: 2 UTF-8 LF HTML Port : 5500
Ln 56, Col 46 Spaces: 2 UTF-8 LF HTML Port : 5500

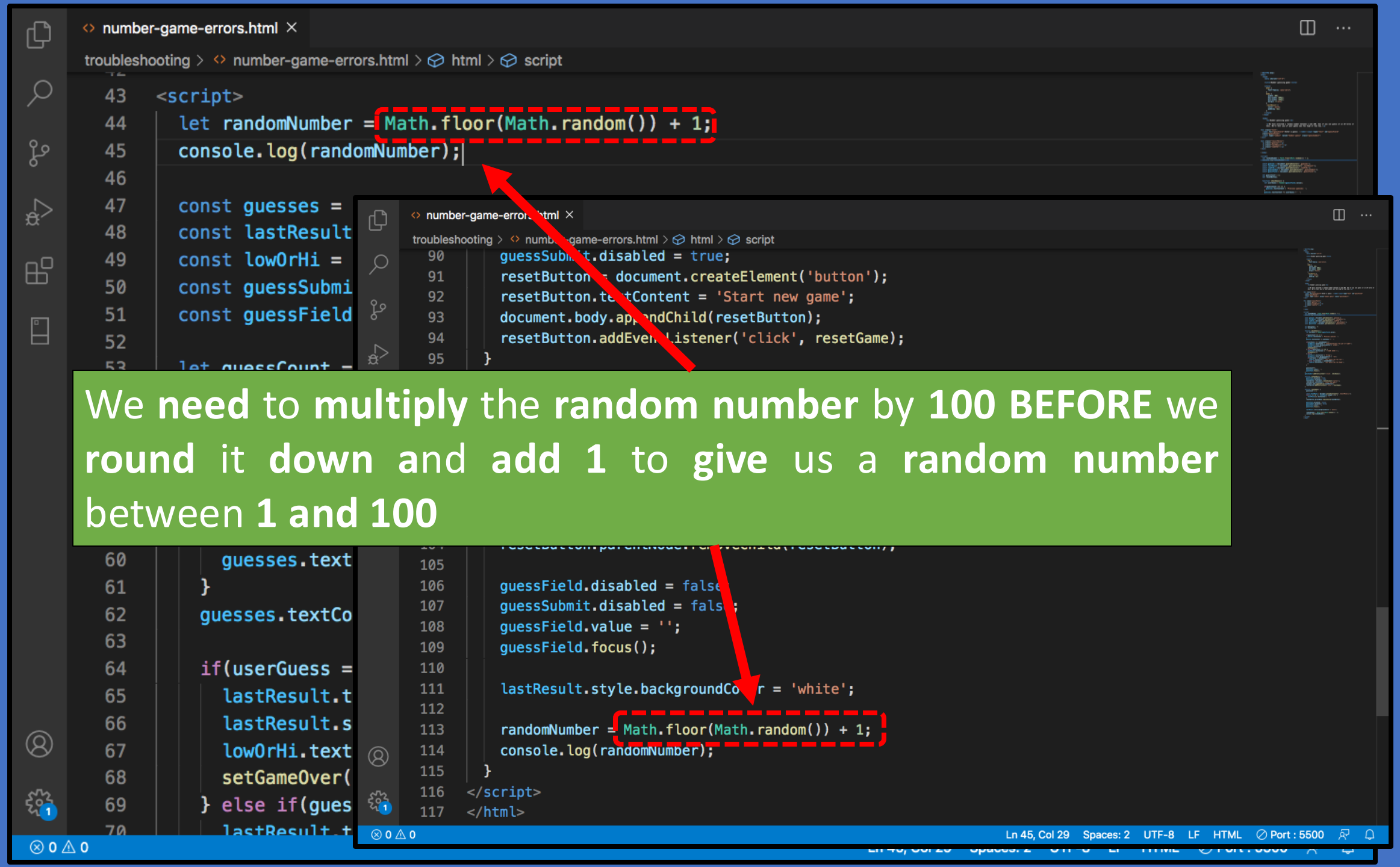

number-game-errors.html
troubleshooting > number-game-errors.html > html > script
43 <script>
44 let randomNumber = Math.floor(Math.random()) + 1;
45 console.log(randomNumber);
Insert the following lines directly below each of the lines with random number manipulation.
102 resetParas[i].textContent = '';
103 }
104 resetButton.parentNode.removeChild(resetButton);
105
106 guessField.disabled = false;
107 guessSubmit.disabled = false;
108 guessField.value = '';
109 guessField.focus();
110
111 lastResult.style.backgroundColor = 'white';
112
113 randomNumber = Math.floor(Math.random()) + 1;
114 console.log(randomNumber);
115 }
116 </script>
117 </html>
Ln 45, Col 29 Spaces: 2 UTF-8 LF HTML Port : 5500

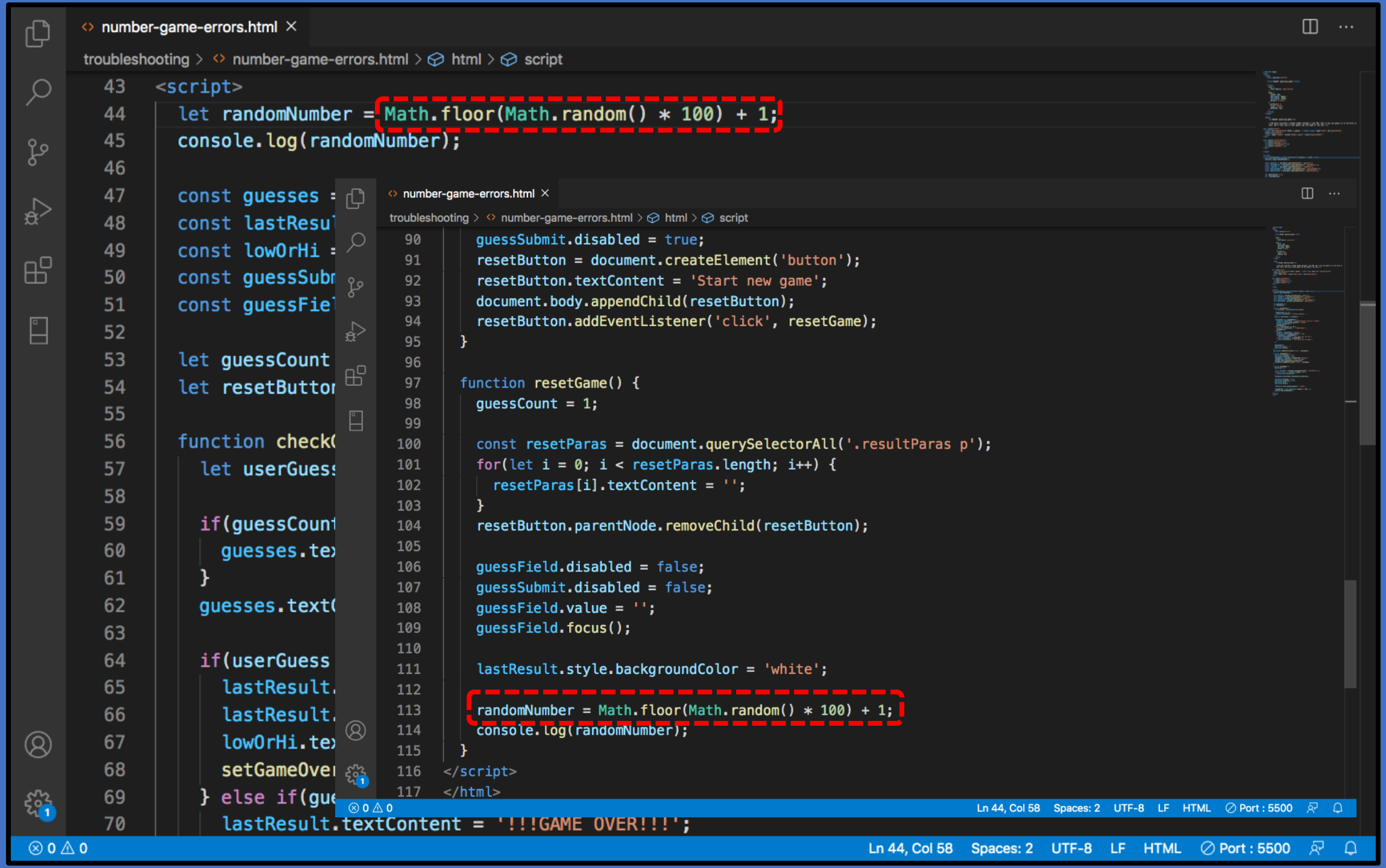

127.0.0.1
Number guessing game
We have selected a random number between 1 and 100. See if you can guess it in 10 turns or less. We'll tell you if your guess was too high or too low.
Enter a guess:
Submit guess
Save and refresh, then play a few games. You'll see that randomNumber is equal to 1 at each point where it is logged to the console.
Elements
Preserve Log
Emulate User Gesture
All
Evaluations
Errors
Warnings
Logs
Console cleared at 5:25:17 PM
1
Global Code — number-game-errors.html:82

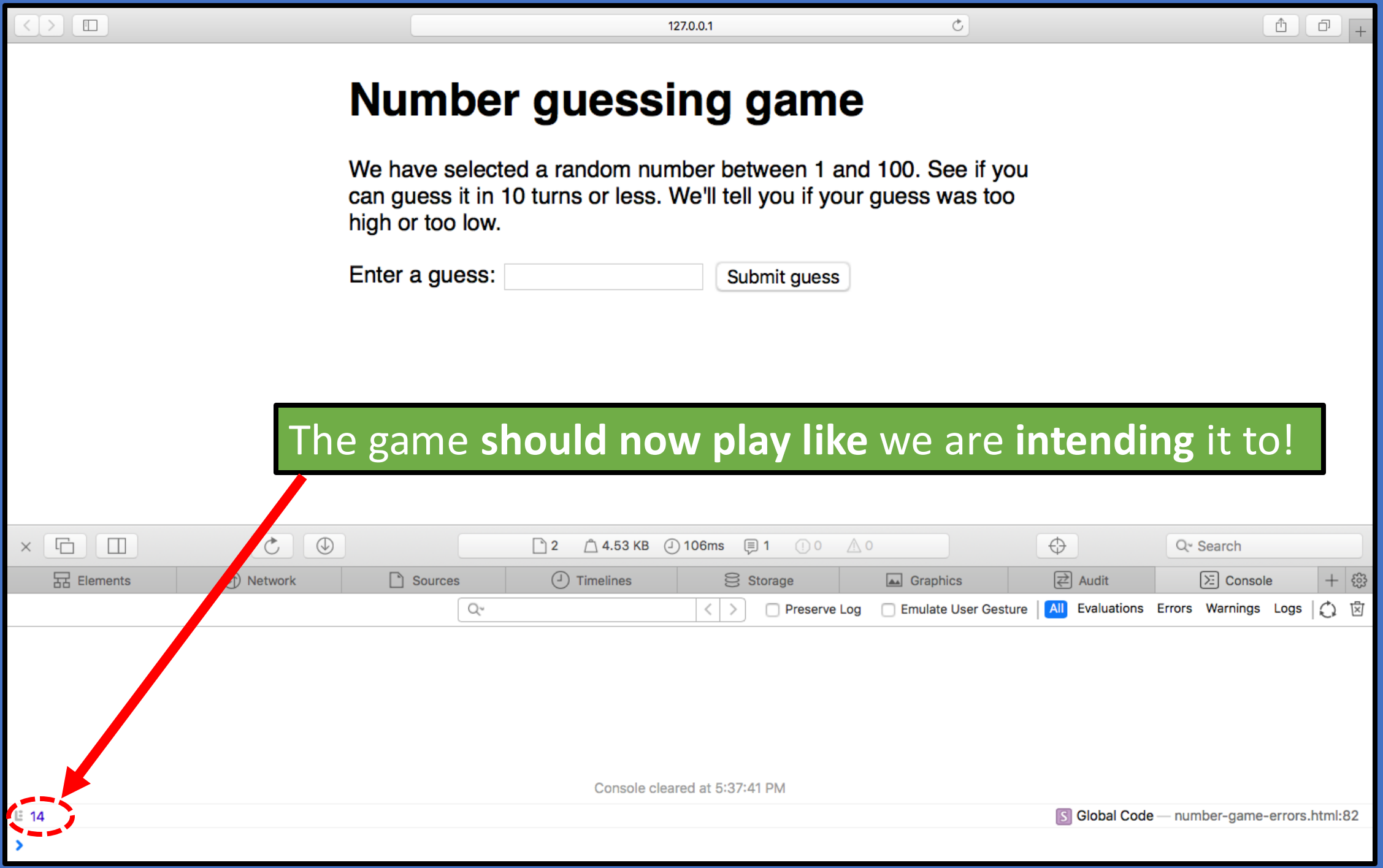

number-game-errors.html
troubleshooting > number-game-errors.html > html > script
<script>
let randomNumber = Math.floor(Math.random()) + 1;
console.log(randomNumber);
Math.random() generates a random decimal number between 0 and 1 (e.g. 0.5675493843)
guessSubmit.disabled = true;
resetButton = document.createElement('button');
resetButton.textContent = 'Start new game';
document.body.appendChild(resetButton);
resetButton.addEventListener('click', resetGame);
resetParas[i].textContent = '';
resetButton.parentNode.removeChild(resetButton);
guessField.disabled = false;
guessSubmit.disabled = false;
guessField.value = '';
guessField.focus();
lastResult.style.backgroundColor = 'white';
randomNumber = Math.floor(Math.random()) + 1;
console.log(randomNumber);
</script>
</html>
Ln 45, Col 29 Spaces: 2 UTF-8 LF HTML Port : 5500


number-game-errors.html
troubleshooting > number-game-errors.html > html > script
<script>
let randomNumber = Math.floor(Math.random()) + 1;
console.log(randomNumber);
Math.floor() rounds the number passed to it down to the nearest whole number. We then add 1 to the result.
randomNumber = Math.floor(Math.random()) + 1;
console.log(randomNumber);
</script>
</html>
Ln 45, Col 29 Spaces: 2 UTF-8 LF HTML Port : 5500


number-game-errors.html
troubleshooting > number-game-errors.html > html > script
<script>
let randomNumber = Math.floor(Math.random()) + 1;
console.log(randomNumber);
Rounding a random decimal number between 0 and 1 down will always return 0, so adding 1 to it will always return 1.
randomNumber = Math.floor(Math.random()) + 1;
console.log(randomNumber);
</script>
</html>
Ln 45, Col 29 Spaces: 2 UTF-8 LF HTML Port : 5500


number-game-errors.html
troubleshooting > number-game-errors.html > html > script
43 <script>
44 let randomNumber = Math.floor(Math.random()) + 1;
45 console.log(randomNumber);
46
47 const guesses =
48 const lastResult
49 const lowOrHi =
50 const guessSubmi
51 const guessField
52
60 guesses.text
61 }
62 guesses.textCo
63
64 if(userGuess =
65 lastResult.t
66 lastResult.s
67 lowOrHi.text
68 setGameOver(
69 } else if(gues
90 guessSubmit.disabled = true;
91 resetButton = document.createElement('button');
92 resetButton.textContent = 'Start new game';
93 document.body.appendChild(resetButton);
94 resetButton.addEventListener('click', resetGame);
95 }
105
106 guessField.disabled = false;
107 guessSubmit.disabled = false;
108 guessField.value = '';
109 guessField.focus();
110
111 lastResult.style.backgroundColor = 'white';
112
113 randomNumber = Math.floor(Math.random()) + 1;
114 console.log(randomNumber);
115 }
116 </script>
117 </html>
Ln 45, Col 29 Spaces: 2 UTF-8 LF HTML Port : 5500
We need to multiply the random number by 100 BEFORE we round it down and add 1 to give us a random number between 1 and 100


number-game-errors.html
troubleshooting > number-game-errors.html > html > script
43 <script>
44 let randomNumber = Math.floor(Math.random() * 100) + 1;
45 console.log(randomNumber);
46
47 const guesses =
48 const lastResu
49 const lowOrHi =
50 const guessSubr
51 const guessFie
52
53 let guessCount
54 let resetButtor
55
56 function check(
57 let userGuess
58
59 if(guessCount
60 guesses.tex
61 }
62 guesses.text(
63
64 if(userGuess
65 lastResult.
66 lastResult.
67 lowOrHi.tex
68 setGameOve
69 } else if(gue
70 lastResult.textContent = '!!!GAME OVER!!!';
number-game-errors.html
troubleshooting > number-game-errors.html > html > script
90 guessSubmit.disabled = true;
91 resetButton = document.createElement('button');
92 resetButton.textContent = 'Start new game';
93 document.body.appendChild(resetButton);
94 resetButton.addEventListener('click', resetGame);
95 }
96
97 function resetGame() {
98 guessCount = 1;
99
100 const resetParas = document.querySelectorAll('.resultParas p');
101 for(let i = 0; i < resetParas.length; i++) {
102 resetParas[i].textContent = '';
103 }
104 resetButton.parentNode.removeChild(resetButton);
105
106 guessField.disabled = false;
107 guessSubmit.disabled = false;
108 guessField.value = '';
109 guessField.focus();
110
111 lastResult.style.backgroundColor = 'white';
112
113 randomNumber = Math.floor(Math.random() * 100) + 1;
114 console.log(randomNumber);
115 }
116 </script>
117 </html>
⊗ 0 ⚠ 0
Ln 44, Col 58 Spaces: 2 UTF-8 LF HTML Port : 5500
⊗ 0 ⚠ 0
Ln 44, Col 58 Spaces: 2 UTF-8 LF HTML Port : 5500


127.0.0.1
Number guessing game
We have selected a random number between 1 and 100. See if you can guess it in 10 turns or less. We'll tell you if your guess was too high or too low.
Enter a guess:
Submit guess
The game should now play like we are intending it to!
2
4.53 KB
106ms
1
0
0
Search
Elements
Network
Sources
Timelines
Storage
Graphics
Audit
Console
Preserve Log
Emulate User Gesture
All
Evaluations
Errors
Warnings
Logs
Console cleared at 5:37:41 PM
14
Global Code — number-game-errors.html:82

Rationale

Rationale

- **When** an **error arises**, **look** at the **line number** you are **given.**
- **Go** to that **line** and **see** if you **can spot** what's **wrong**.
- **Bear** in **mind** that the **error** is **not necessarily going** to **be** on **that line**.

Questions?